ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KINH TẾ - LUẬT

KHOA HỆ THỐNG THÔNG TIN

BÀI TẬP QUÁ TRÌNH

HỌC PHẦN: HỌC MÁY TRONG PHÂN TÍCH KINH DOANH

GOM CỤM KHÁCH HÀNG CHINOOK THEO MỨC ĐỘ QUAN TÂM TRACK VÀ ALBUM BẰNG K-MEANS

GVHD: TS. Trần Duy Thanh
SVTH: Đỗ Lê Quỳnh Kha
MSSV: K244161790

TP.HCM, THÁNG 10/2026

CHƯƠNG 1: MÔ TẢ BÀI TOÁN & LÝ DO LỰA CHỌN

Bài làm chọn bài toán thứ ba: gom cụm 59 khách hàng của cửa hàng nhạc Chinook theo mức độ quan tâm tới track và album. Lựa chọn này dựa trên kết quả khám phá dữ liệu, không phải chọn ngẫu nhiên. Trước khi quyết định, cả ba bài toán đề xuất đều được đối chiếu với dữ liệu thực tế trong file Chinook_Sqlite.sqlite.

1.1. Phát biểu bài toán

Đầu vào là lịch sử mua hàng của từng khách, gồm các track đã mua và album, thể loại, thời lượng tương ứng của mỗi track. Đầu ra là nhãn cụm cho mỗi khách, sao cho khách trong cùng một cụm có hành vi quan tâm gần nhau. Đây là bài toán học không giám sát: dữ liệu không có sẵn nhãn "khách thuộc nhóm nào", nên mô hình phải tự tìm ra các nhóm.

Ý nghĩa kinh doanh của kết quả là phân khúc khách hàng. Khi biết mỗi nhóm quan tâm loại nội dung nào, cửa hàng có thể thiết kế khuyến mãi, danh sách phát và gợi ý album riêng cho từng nhóm thay vì dùng một chiến dịch chung cho tất cả.

1.2. Vì sao không chọn bài toán dự báo doanh thu

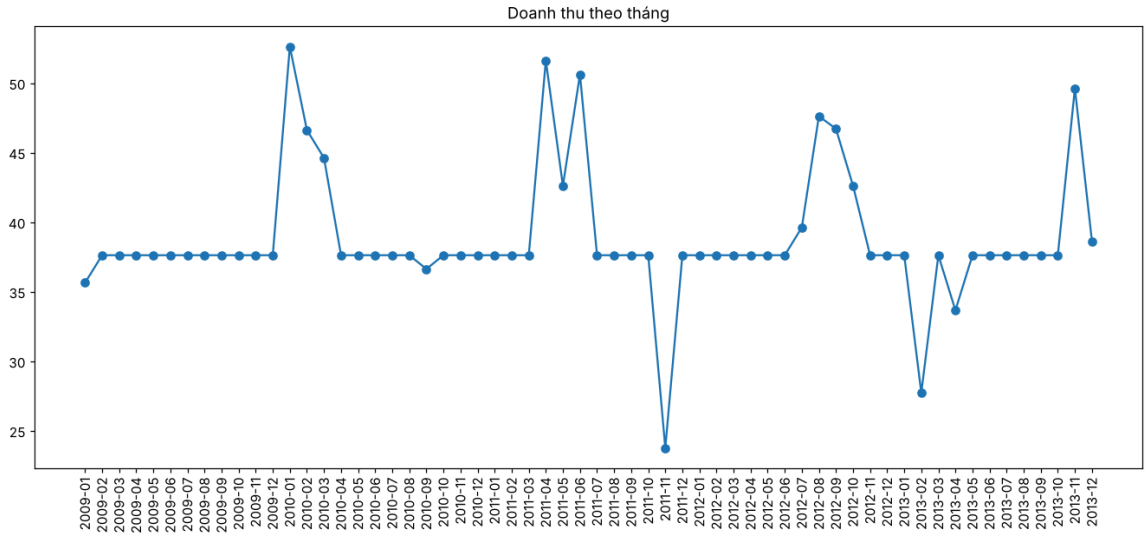
Doanh thu của Chinook gần như không biến động. Trong 60 tháng từ 01/2009 đến 12/2013, có 43 tháng doanh thu đúng bằng 37,62 USD. Trung vị, tứ phân vị thứ nhất và tứ phân vị thứ ba đều là 37,62 USD; mỗi năm có 80–83 hóa đơn. Đây là dấu hiệu của dữ liệu được sinh theo một chu kỳ cố định. Một mô hình dự báo trên chuỗi này chỉ tái hiện lại hằng số 37,62, không mang lại thông tin mới cho việc ra quyết định.

```
14  # 2. Doanh thu theo tháng -> hầu như không đổi (dữ liệu giả lập)
15  #    => bài toán dự báo doanh thu không có nhiều ý nghĩa
16  sql2 = "select strftime('%Y-%m', InvoiceDate) as Month, round(sum(Total), 2) as Revenue " \
17         "from Invoice group by Month"
18  df2 = queryDataset(conn, sql2)
19  print(df2.head())
20  print(df2["Revenue"].describe())
21  print("Số tháng có doanh thu đúng 37.62:", (df2["Revenue"] == 37.62).sum(), "/", len(df2))
22
```

Hình 1.1. Đoạn mã tính doanh thu theo tháng

```
count    60.000000
mean     38.810000
std       4.743155
min      23.760000
25%      37.620000
50%      37.620000
75%      37.620000
max      52.620000
Name: Revenue, dtype: float64
Số tháng có doanh thu đúng 37.62: 43 / 60
```

Hình 1.2. Kết quả chạy: 43/60 tháng có doanh thu đúng 37,62 USD.



Hình 1.3. Doanh thu theo tháng 2009–2013: đường gần như nằm ngang, chỉ vài tháng lệch khỏi 37,62 USD.

1.3. Vì sao không chọn bài toán gợi ý track, album

Dữ liệu mua hàng quá thưa để đánh giá một hệ gợi ý. Bảng InvoiceLine có 2.240 dòng, trải trên 1.984 track khác nhau. Trong đó 1.728 track chỉ được bán đúng một lần, 256 track được bán hai lần, và không khách nào mua lại track mình đã mua. Mỗi khách mua khoảng 38 track, tức ma trận khách × track chỉ có khoảng 1,9% ô khác 0. Với mức thưa này, gần như không có cặp khách nào cùng mua một track, nên không thể tách tập kiểm tra để đo độ chính xác của gợi ý.

1.4. Vì sao bài toán gom cụm phù hợp

Các chỉ số về *số lượng* mua gần như giống nhau giữa các khách, nhưng các chỉ số về *nội dung* mua lại khác nhau rõ rệt. Mỗi khách có 6–7 hóa đơn và 36–38 track, nhưng số thể loại đã mua dao động từ 4 đến 12, số album từ 18 đến 27 (Hình 2.2). Sự khác biệt nằm ở *khách quan tâm cái gì*, không phải *khách mua bao nhiêu*. Đây đúng là loại khác biệt mà gom cụm có thể khai thác.

Hướng tiếp cận cũng bám theo Bài tập 2 ở bài giảng 07: gom cụm khách hàng cơ sở dữ liệu sakila theo mức độ quan tâm Film và Inventory, trong đó sinh viên tự đề xuất thuộc tính và bảo vệ ý tưởng của mình. Bài này áp dụng cách làm đó cho Chinook: "track" và "album" đóng vai trò của "film" và "inventory".

CHƯƠNG 2. CƠ SỞ DỮ LIỆU CHINOOK VÀ KẾT QUẢ KHÁM PHÁ

Chinook là cơ sở dữ liệu mẫu mô phỏng một cửa hàng bán nhạc số. File SQLite gồm 11 bảng; bài làm sử dụng 7 bảng nằm trên chuỗi "khách hàng → hóa đơn → track → album/thể loại". Các bảng Employee, MediaType, Playlist và PlaylistTrack không dùng, vì chúng không phản ánh hành vi mua của khách.

```
Album : 347 dòng
Artist : 275 dòng
Customer : 59 dòng
Employee : 8 dòng
Genre : 25 dòng
Invoice : 412 dòng
InvoiceLine : 2240 dòng
MediaType : 5 dòng
Playlist : 18 dòng
PlaylistTrack : 8715 dòng
Track : 3503 dòng
```

Hình 2.1. Danh sách 11 bảng và số dòng, in từ Clustering/data_exploration.py.

2.1. Các bảng và cột được sử dụng

Bảng	Số dòng	Cột sử dụng	Vai trò trong bài
Customer	59	CustomerId, FirstName, LastName, Country, City, Email	Đối tượng được gom cụm; hiển thị danh sách chi tiết theo cụm
Invoice	412	InvoiceId, CustomerId, InvoiceDate, Total	Nối khách với các dòng hóa đơn; tính doanh thu theo tháng
InvoiceLine	2.240	InvoiceId, TrackId, UnitPrice	Mỗi dòng là một track mà khách đã mua
Track	3.503	TrackId, AlbumId, GenreId, Milliseconds	Cung cấp album, thể loại và thời lượng của track
Album	347	AlbumId, Title, ArtistId	Tính số album và độ phổ biến album; tên album khi gợi ý
Artist	275	ArtistId, Name	Tên nghệ sĩ trong bảng gợi ý album
Genre	25	GenreId, Name	Thống kê thể loại bán chạy

Quan hệ giữa các bảng đều là một–nhiều: một khách có nhiều hóa đơn, một hóa đơn có nhiều dòng, mỗi dòng trỏ tới một track, mỗi track thuộc một album và một thể loại. Vì

vậy câu truy vấn nối Invoice → InvoiceLine → Track trả về đúng 2.240 dòng, mỗi dòng là một lượt mua track của một khách.

2.2. Hành vi mua của khách hàng

Bảng thống kê ở Hình 2.2 cho thấy ba điểm quyết định cách thiết kế thuộc tính ở Chương 4:

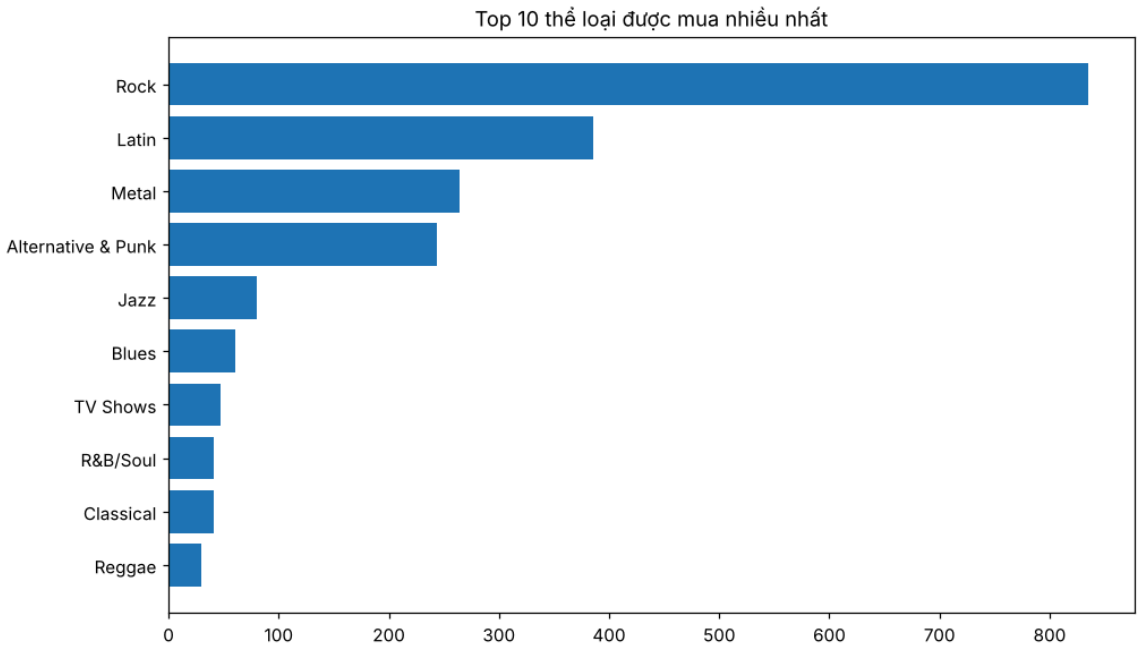
1. **InvoiceCount và TrackCount gần như là hằng số.** Độ lệch chuẩn lần lượt là 0,13 và 0,26; 58/59 khách có đúng 7 hóa đơn và ít nhất 75% khách mua đúng 38 track. Hai cột này không phân biệt được khách nên bị loại khỏi tập thuộc tính.
2. **GenreCount và AlbumCount có độ phân tán đáng kể.** GenreCount từ 4 đến 12 (độ lệch chuẩn 1,71), AlbumCount từ 18 đến 27 (độ lệch chuẩn 2,19). Đây là nơi các khách thực sự khác nhau.
3. **Track hầu như không lặp lại giữa các khách.** 1.728 track chỉ bán một lần, nên không thể dùng trực tiếp từng track làm thuộc tính. Thông tin về track phải được tổng hợp lên mức thể loại và thời lượng.

```
       CustomerId  InvoiceCount  TrackCount  GenreCount  AlbumCount
count   59.000000     59.000000   59.000000   59.000000   59.000000
mean    30.000000      6.983051   37.966102    7.457627   22.050847
std     17.175564      0.130189    0.260378    1.705185    2.192650
min      1.000000      6.000000   36.000000    4.000000   18.000000
25%     15.500000      7.000000   38.000000    6.000000   20.000000
50%     30.000000      7.000000   38.000000    7.000000   22.000000
75%     44.500000      7.000000   38.000000    9.000000   24.000000
max     59.000000      7.000000   38.000000   12.000000   27.000000
TimesSold
1    1728
2     256
Name: count, dtype: int64
```

Hình 2.2. Thống kê mô tả số hóa đơn, số track, số thể loại, số album theo khách và số lần bán của mỗi track.

2.3. Thể loại được mua nhiều nhất

Rock chiếm 835/2.240 lượt mua (37,3%), tiếp theo là Latin (386), Metal (264) và Alternative & Punk (244). Bốn thể loại này chiếm 77% tổng lượt mua. Các thể loại dạng video (TV Shows, Drama, Sci Fi & Fantasy, Science Fiction, Comedy) chỉ có 111 lượt mua. Tuy vậy, thời lượng trung bình mỗi track của chúng từ 27,7 đến 48,5 phút, trong khi track nhạc chỉ khoảng 4,5 phút. Nhờ vậy thuộc tính thời lượng trung bình ở Chương 4 tách được nhóm khách hay mua nội dung video.



Hình 2.3. Top 10 thể loại theo số track đã bán.

CHƯƠNG 3. THIẾT KẾ CHƯƠNG TRÌNH

Chương trình được tổ chức theo cấu trúc của các bài thực hành trên lớp và chia thành sáu thư mục con (Hình 3.1). Dataset/ chứa file SQLite. Utils/ chứa ba thư viện dùng chung: ClusterUtil.py cho phần máy học, ExportUtil.py cho phần xuất kết quả và FileUtil.py giữ nguyên từ bài Housing price prediction. Clustering/ chứa các file chạy theo đúng thứ tự các bước làm bài, từ khám phá dữ liệu đến xuất trang HTML. Ui/ chứa giao diện, tách riêng giống SalesDatabaseProject. Trained/ chứa mô hình đã lưu. results/ chứa kết quả xuất ra: ba file Excel, trang index.html và 25 ảnh chụp từ trang này.

Mọi đường dẫn đều được tính từ vị trí của ClusterUtil.py (biến BASE_DIR, DATABASE_PATH, TRAINED_DIR, EXCEL_DIR, HTML_DIR, IMAGES_DIR). Nhờ vậy chạy file ở thư mục nào cũng đọc đúng cơ sở dữ liệu và ghi đúng chỗ. Các thư mục là thư mục thường, không có file __init__.py; mọi file import bằng tên đầy đủ, ví dụ from Chinook_Customer_Clustering.Utils.ClusterUtil import *, để PyCharm nhận đúng khi gốc project là D:\ProjectML.

```
Chinook_Customer_Clustering/
├── Dataset/
│   └── Chinook_Sqlite.sqlite          (cơ sở dữ liệu Chinook)
├── Utils/                             (thư viện dùng chung)
│   ├── ClusterUtil.py                 (kết nối, thuộc tính, Elbow + Kneedle, K-Means, tên cụm)
│   ├── ExportUtil.py                  (xuất Excel và trang HTML)
│   └── FileUtil.py                    (lưu / tải mô hình)
├── Clustering/                        (các bước máy học)
│   ├── data_exploration.py            (khám phá dữ liệu)
│   ├── CustomerCluster1.py            (thực nghiệm 1: quan tâm TRACK)
│   ├── CustomerCluster2.py            (thực nghiệm 2: quan tâm ALBUM)
│   ├── CustomerCluster3.py            (thực nghiệm 3: TRACK + ALBUM, 3D)
│   ├── using_model.py                 (sử dụng mô hình trên console)
│   └── export_results.py              (xuất trang HTML từ 3 mô hình đã lưu)
├── Ui/
│   ├── ChinookMainWindow.ui           (thiết kế bằng Qt Designer)
│   ├── ChinookMainWindow.py           (sinh tự động bằng PyUIC)
│   └── ChinookMainWindowEx.py         (xử lý sự kiện, tích hợp mô hình)
├── Trained/                           (6 file mô hình *_model.zip, *_scaler.zip)
├── results/
│   ├── excel/                         (3 file Excel, mỗi cụm 1 sheet + sheet Thống kê)
│   ├── html/index.html                (trang kết quả 5 tab, biểu đồ plotly)
│   └── images/                        (25 ảnh chụp từ index.html)
├── main.py                            (chạy giao diện PyQt6)
├── requirements.txt
└── README.md
```

Hình 3.1. Cấu trúc thư mục Chinook_Customer_Clustering sau khi sắp xếp lại.

3.1. Vai trò của từng file

File	Nội dung	Bước tương ứng (bài giảng 08)
Utils/ClusterUtil.py	Đường dẫn dùng chung, kết nối, truy vấn, tạo thuộc tính, chuẩn hóa, Elbow + Kneedle, K-Means, trực quan hóa, đặt tên cụm, gợi ý album	6.4 Thư viện phụ trợ, 6.6 Tiền xử lý
Utils/ExportUtil.py	Xuất kết quả gom cụm ra file Excel và trang HTML	6.11 Sử dụng mô hình
Utils/FileUtil.py	Lưu và tải mô hình bằng pickle	6.10 Lưu mô hình
Clustering/data_exploration.py	Thống kê bảng, doanh thu, hành vi mua, thể loại	6.3 Thu thập và mô tả dữ liệu
Clustering/CustomerCluster1.py	Thực nghiệm 1: GenreCount × AvgMinutes, K = 3; xuất CustomerCluster1_Track.xlsx	6.7–6.10 Chọn giải thuật, train, đánh giá, lưu
Clustering/CustomerCluster2.py	Thực nghiệm 2: AlbumCount × AlbumPopularity, K = 3; xuất CustomerCluster2_Album.xlsx	6.7–6.10
Clustering/CustomerCluster3.py	Thực nghiệm 3: ba thuộc tính, K = 4, biểu đồ 3D; xuất CustomerCluster3_TrackAlbum.xlsx	6.7–6.10
Clustering/using_model.py	Tải mô hình, xác định cụm của khách, gợi ý album, dự đoán khách mới	6.11 Sử dụng mô hình
Clustering/export_results.py	Tải ba mô hình đã lưu, tạo trang results/html/index.html	6.11 Sử dụng mô hình
Ui/ChinookMainWindow.ui, .py	Giao diện thiết kế bằng Qt Designer và file sinh từ PyUIC	6.5 Màn hình giao diện
Ui/ChinookMainWindowEx.py, main.py	Xử lý sự kiện, tích hợp mô hình vào giao diện	6.11 Sử dụng mô hình

File	Nội dung	Bước tương ứng (bài giảng 08)
results/excel, html, images	3 file Excel, trang HTML 5 tab, 25 ảnh chụp từ trang HTML	Kết quả nộp kèm

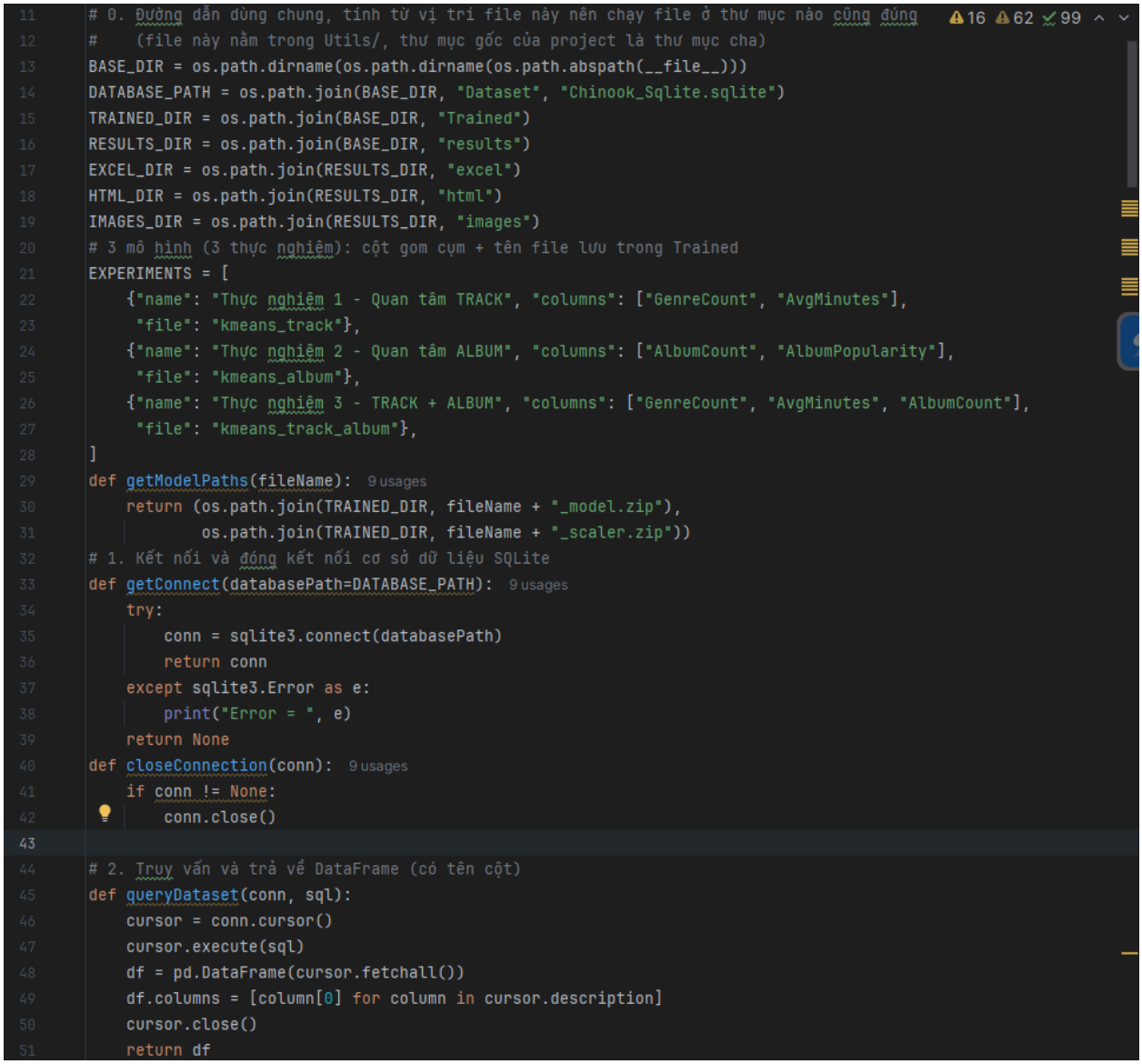
3.2. Các hàm trong ClusterUtil.py

Tên và đối số của các hàm getConnect, closeConnection, queryDataset, showHistogram, elbowMethod, runKMeans, visualizeKMeans, visualize3DKmeans giữ đúng theo case study CustomerCluster1.py trong bài giảng 07; kết nối chuyển từ MySQL sang SQLite. Các hàm còn lại được viết thêm cho bài toán Chinook: nhóm lấy dữ liệu và tạo thuộc tính, findElbow cài đặt thuật toán Kneedle (bài học bổ sung về K-Means), nameClusters đặt tên cụm tự động, recommendAlbums gợi ý album và getClusterDetail gom thông tin khách theo cụm để xuất file. showCustomerDetailByCluster giải Bài tập 1 của bài giảng 07. Hai hàm xuất kết quả nằm riêng trong ExportUtil.py vì chúng không thuộc bước máy học.

Hàm	Đầu vào	Đầu ra	Ghi chú
getModelPaths	tên mô hình, ví dụ "kmeans_track"	đường dẫn file mô hình và file chuẩn hóa trong Trained/	Dùng chung cho console, HTML và giao diện
getConnect	đường dẫn file .sqlite (mặc định DATABASE_PATH)	đối tượng kết nối	Bắt lỗi sqlite3.Error
queryDataset	kết nối, câu SQL	DataFrame có tên cột	Tên cột lấy từ cursor.description
getPurchaseDataset	kết nối	2.240 dòng (khách, track, album, thể loại, thời lượng)	Nối Invoice, InvoiceLine, Track
getCustomerInfo	kết nối	59 dòng: họ tên, quốc gia, thành phố, email, tổng chi tiêu	Dùng cho bảng chi tiết và trang tra cứu

Hàm	Đầu vào	Đầu ra	Ghi chú
getAlbumPurchaseDataset	kết nối	lượt mua theo khách, album, nghệ sĩ, thể loại	Dữ liệu cho gợi ý album và top album mỗi cụm
getCustomerFeatures	kết nối	59 dòng × 5 cột	Trình bày ở Chương 4
minMaxScale	DataFrame, danh sách cột	dữ liệu chuẩn hóa, min, max	Trả min, max để chuẩn hóa khách mới
findElbow	ma trận X, maxK = 10, tolerance = 0,9	WCSS theo K, khoảng elbow, K tối ưu	Thuật toán Kneedle, mục 5.4
elbowMethod	DataFrame, danh sách cột	đồ thị Elbow có tô khoảng elbow; trả về K	Gọi findElbow; init='k-means++', max_iter=500, random_state=42
runKMeans	ma trận X, số cụm	y_kmeans, centroids, labels, model	Trả thêm model để lưu xuống đĩa
visualizeKMeans	X, nhãn, số cụm, tiêu đề, nhãn trục, màu	biểu đồ phân tán 2D	Vẽ bằng giá trị gốc để trục dễ đọc
visualize3DKmeans	DataFrame, 3 cột, hover_data, số cụm	biểu đồ 3D plotly	Mở trên trình duyệt
nameClusters	DataFrame đã gán cụm, danh sách cột	tên mô tả của từng cụm	Dựa trên z-score của trung bình cụm (mục 6.4)
showClusterSummary	DataFrame đã gán cụm, danh sách cột	trung bình thuộc tính và số khách mỗi cụm	Cụm đánh số từ 1
showCustomerDetailByCluster	kết nối, DataFrame, số cụm	danh sách chi tiết khách theo cụm	Bài tập 1, bài giảng 07

Hàm	Đầu vào	Đầu ra	Ghi chú
recommendAlbums	lượt mua album, khách cùng cụm, CustomerId, topN	top N album khách chưa mua	Sắp theo số khách giảm dần, rồi theo tên album
getClusterDetail	kết nối, DataFrame đã gán cụm, danh sách cột	bảng khách kèm cụm và tên cụm	Đầu vào cho Excel và HTML
exportClustersToExcel (ExportUtil)	bảng chi tiết, danh sách cột, tên file	file .xlsx	Mỗi cụm một sheet, thêm sheet Thống kê
exportClustersToHtml (ExportUtil)	tổng quan, ba thực nghiệm, bảng tra cứu, tên file	results/html/index.html	5 tab, biểu đồ plotly nhúng sẵn



```python
# 0. Đường dẫn dùng chung, tính từ vị trí file này nên chạy file ở thư mục nào cũng đúng
#    (file này nằm trong Utils/, thư mục gốc của project là thư mục cha)
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATABASE_PATH = os.path.join(BASE_DIR, "Dataset", "Chinook_Sqlite.sqlite")
TRAINED_DIR = os.path.join(BASE_DIR, "Trained")
RESULTS_DIR = os.path.join(BASE_DIR, "results")
EXCEL_DIR = os.path.join(RESULTS_DIR, "excel")
HTML_DIR = os.path.join(RESULTS_DIR, "html")
IMAGES_DIR = os.path.join(RESULTS_DIR, "images")
# 3 mô hình (3 thực nghiệm): cột gom cụm + tên file lưu trong Trained
EXPERIMENTS = [
    {"name": "Thực nghiệm 1 - Quan tâm TRACK", "columns": ["GenreCount", "AvgMinutes"],
     "file": "kmeans_track"},
    {"name": "Thực nghiệm 2 - Quan tâm ALBUM", "columns": ["AlbumCount", "AlbumPopularity"],
     "file": "kmeans_album"},
    {"name": "Thực nghiệm 3 - TRACK + ALBUM", "columns": ["GenreCount", "AvgMinutes", "AlbumCount"],
     "file": "kmeans_track_album"},
]
def getModelPaths(fileName):  9 usages
    return (os.path.join(TRAINED_DIR, fileName + "_model.zip"),
            os.path.join(TRAINED_DIR, fileName + "_scaler.zip"))
# 1. Kết nối và đóng kết nối cơ sở dữ liệu SQLite
def getConnect(databasePath=DATABASE_PATH):  9 usages
    try:
        conn = sqlite3.connect(databasePath)
        return conn
    except sqlite3.Error as e:
        print("Error = ", e)
    return None
def closeConnection(conn):  9 usages
    if conn != None:
        conn.close()

# 2. Truy vấn và trả về DataFrame (có tên cột)
def queryDataset(conn, sql):
    cursor = conn.cursor()
    cursor.execute(sql)
    df = pd.DataFrame(cursor.fetchall())
    df.columns = [column[0] for column in cursor.description]
    cursor.close()
    return df
```

Hình 3.2. Đường dẫn dùng chung, danh sách ba thực nghiệm, hàm getModelPaths và các hàm kết nối, truy vấn SQLite

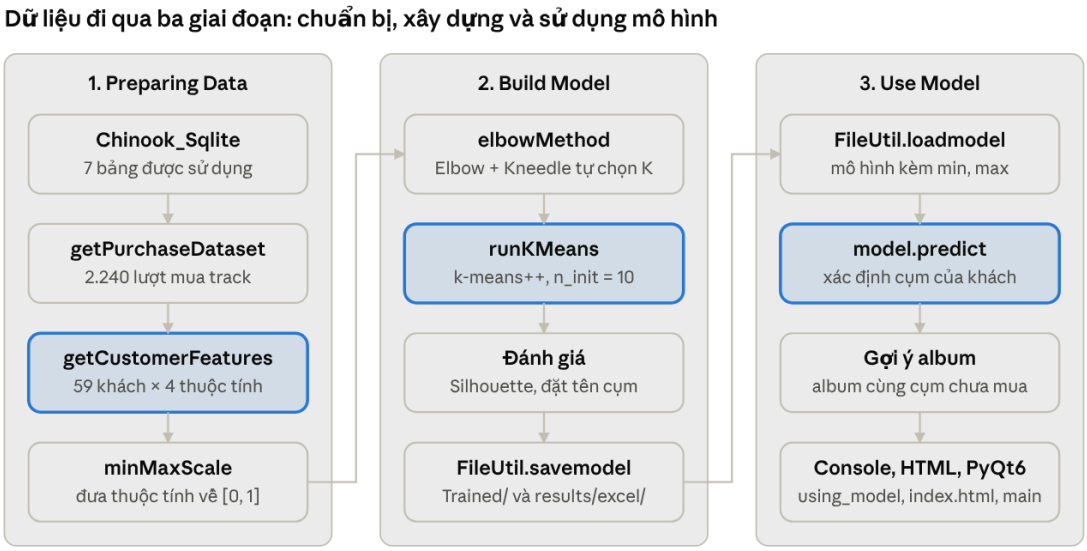
Hàm runKMeans trong bài giảng gọi model.fit(X) rồi gọi tiếp model.fit_predict(X), tức huấn luyện hai lần. Bài làm thay lần thứ hai bằng model.predict(X). Cách này cho cùng kết quả nhưng không huấn luyện lại, đảm bảo labels và y_kmeans luôn khớp với mô hình được lưu. Tham số n_init=10 cũng được ghi rõ. Từ scikit-learn 1.4, giá trị mặc định của n_init đổi thành 'auto' (chạy một lần với k-means++). Không ghi rõ tham số này thì kết quả có thể khác nhau giữa các máy cài phiên bản khác nhau.

```python
def runKMeans(X, cluster):  6 usages
    model = KMeans(n_clusters=cluster,
                   init='k-means++',
                   max_iter=500,
                   n_init=10,
                   random_state=42)
    model.fit(X)
    labels = model.labels_
    centroids = model.cluster_centers_
    y_kmeans = model.predict(X)
    return y_kmeans, centroids, labels, model
```

Hình 3.3. Hàm runKMeans

3.3. Luồng xử lý

Toàn bộ hệ thống đi theo ba giai đoạn: Preparing Data, Build Model và Use Model.



Hình 3.4. Luồng xử lý của hệ thống 3 giai đoạn, 12 bước

Hai bước tô màu ở cột giữa và cột cuối là nơi mô hình được tạo ra và được dùng. Bước tô màu ở cột đầu là nơi thiết kế thuộc tính, quyết định chất lượng của cả hai bước sau. Kết quả được dùng ở ba nơi: console, trang HTML và giao diện PyQt6.

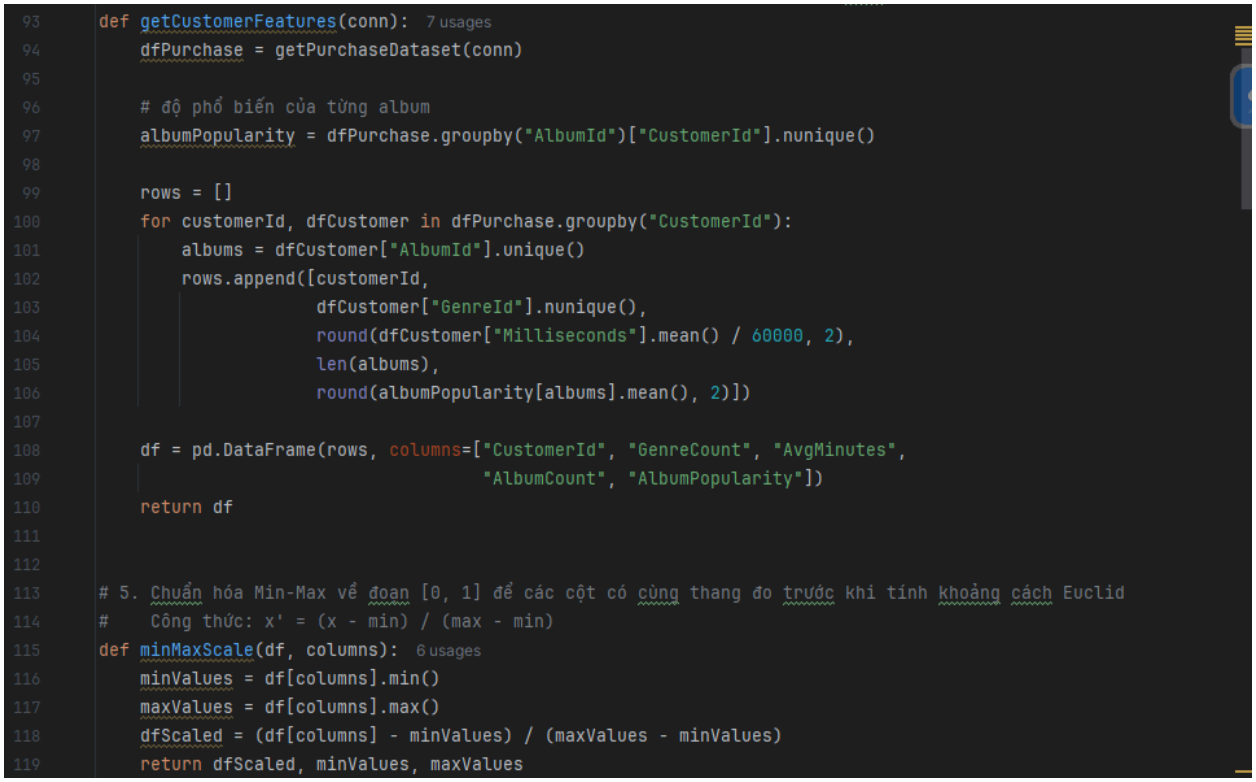
CHƯƠNG 4. TIỀN XỬ LÝ VÀ ĐỀ XUẤT THUỘC TÍNH

Mỗi khách được biểu diễn bằng bốn thuộc tính: hai thuộc tính đo mức độ quan tâm track, hai thuộc tính đo mức độ quan tâm album. Cả bốn được tính trong hàm getCustomerFeatures từ 2.240 lượt mua, sau đó chuẩn hóa Min-Max trước khi đưa vào K-Means. Dữ liệu không có giá trị thiếu: mọi dòng InvoiceLine đều nối được tới Track, và mọi Track đều có AlbumId, GenreId, Milliseconds.

4.1. Định nghĩa thuộc tính

Thuộc tính	Cách tính	Ý nghĩa	Khoảng giá trị
GenreCount	Số GenreId khác nhau trong các track khách đã mua	Khách nghe đa dạng hay chỉ tập trung vào vài thể loại	4–12
AvgMinutes	Trung bình Milliseconds của các track đã mua, chia 60.000	Khách mua bài hát ngắn hay nội dung video dài	3,56–16,55 phút
AlbumCount	Số AlbumId khác nhau trong các track đã mua	Khách mua rải rác nhiều album hay gom vào ít album	18–27
AlbumPopularity	Trung bình độ phổ biến của các album khách đã mua; độ phổ biến của một album là số khách đã mua ít nhất một track của album đó	Khách theo album được nhiều người mua hay album ít người biết	3,68–6,43

Để minh họa, khách số 12 (Roberto Almeida, Brazil) mua 38 track. Trong đó 16 track Rock, 16 track Latin và mỗi thể loại Metal, World, Alternative & Punk có 2 track, nên GenreCount = 5. Thời lượng trung bình 38 track là 3,82 phút. 38 track này thuộc 18 album, nên AlbumCount = 18. 18 album đó có độ phổ biến từ 4 đến 11 khách, trung bình 5,83, nên AlbumPopularity = 5,83. Toàn bộ 304 album từng được bán có độ phổ biến từ 1 đến 15 khách, trung bình 4,28. Vậy khách 12 thiên về các album được nhiều người mua.



```
def getCustomerFeatures(conn):  7 usages
    dfPurchase = getPurchaseDataset(conn)

    # độ phổ biến của từng album
    albumPopularity = dfPurchase.groupby("AlbumId")["CustomerId"].nunique()

    rows = []
    for customerId, dfCustomer in dfPurchase.groupby("CustomerId"):
        albums = dfCustomer["AlbumId"].unique()
        rows.append([customerId,
                     dfCustomer["GenreId"].nunique(),
                     round(dfCustomer["Milliseconds"].mean() / 60000, 2),
                     len(albums),
                     round(albumPopularity[albums].mean(), 2)])

    df = pd.DataFrame(rows, columns=["CustomerId", "GenreCount", "AvgMinutes",
                                     "AlbumCount", "AlbumPopularity"])
    return df


# 5. Chuẩn hóa Min-Max về đoạn [0, 1] để các cột có cùng thang đo trước khi tính khoảng cách Euclid
#    Công thức: x' = (x - min) / (max - min)
def minMaxScale(df, columns):  6 usages
    minValues = df[columns].min()
    maxValues = df[columns].max()
    dfScaled = (df[columns] - minValues) / (maxValues - minValues)
    return dfScaled, minValues, maxValues
```

Hình 4.1. Hàm getCustomerFeatures và minMaxScale

4.2. Lý do chọn và loại thuộc tính

Các thuộc tính được chọn theo hai tiêu chí: phải có độ phân tán giữa các khách, và phải diễn giải được thành hành vi kinh doanh.

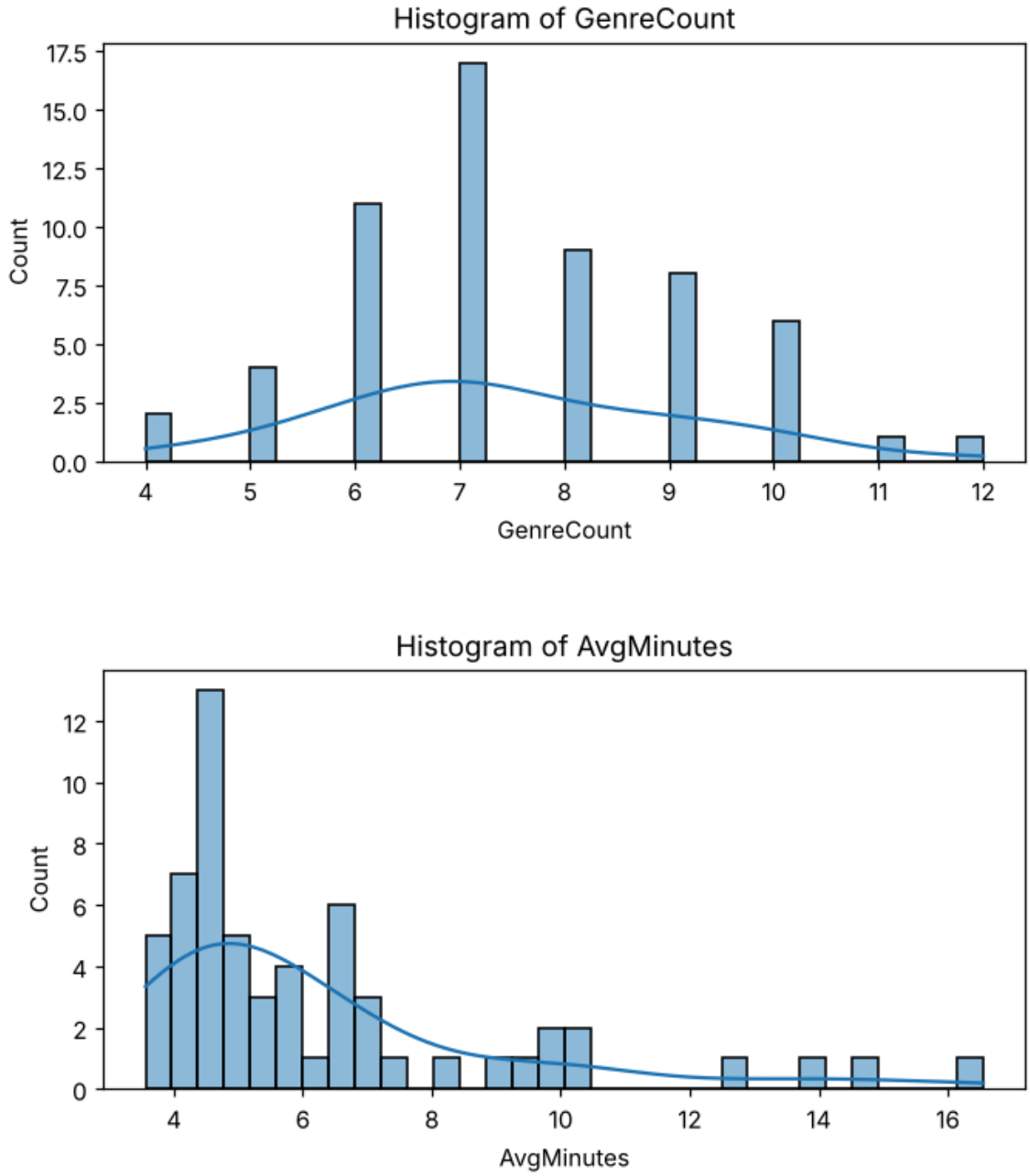
- **Loại TrackCount và InvoiceCount.** Hai cột này gần như bằng nhau ở mọi khách (Chương 2), nên không giúp phân biệt khách.
- **Loại tổng chi tiêu.** Giá track nhạc là 0,99 USD, track video là 1,99 USD. Do đó tổng chi tiêu chỉ phản ánh số track video đã mua: hệ số tương quan giữa tổng chi tiêu và AvgMinutes là 0,97. Giữ cả hai cột sẽ đếm một thông tin hai lần.
- **Không dùng ma trận khách × track hoặc khách × album dạng 0/1.** Ma trận khách × album có 304 cột nhưng chỉ 7,3% ô bằng 1. Hai khách bất kỳ trung bình chỉ chung 1,6 album. Khi thử K-Means trực tiếp trên ma trận này, hệ số Silhouette chỉ đạt 0,03–0,06 với k từ 2 đến 8, tức các cụm gần như ngẫu nhiên. Bốn thuộc tính tổng hợp ở bảng trên giữ được thông tin về track và album nhưng ngắn gọn hơn nhiều.

	CustomerId	GenreCount	AvgMinutes	AlbumCount	AlbumPopularity
count	59.000000	59.000000	59.000000	59.000000	59.000000
mean	30.000000	7.457627	6.256949	22.050847	5.216271
std	17.175564	1.705185	2.852181	2.192650	0.532240
min	1.000000	4.000000	3.560000	18.000000	3.680000
25%	15.500000	6.000000	4.470000	20.000000	5.000000
50%	30.000000	7.000000	5.160000	22.000000	5.230000
75%	44.500000	9.000000	6.705000	24.000000	5.565000
max	59.000000	12.000000	16.550000	27.000000	6.430000

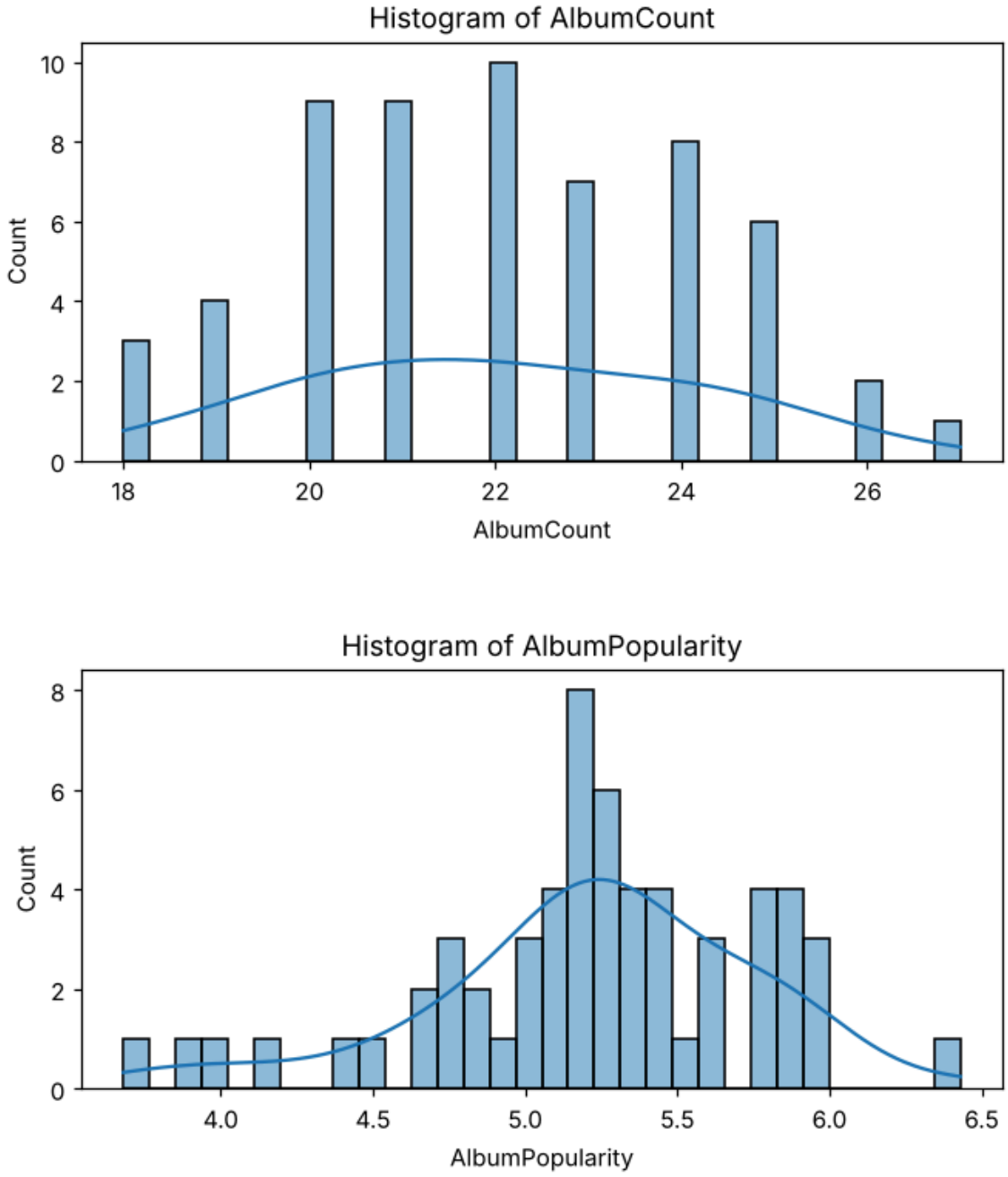
Hình 4.2. Thống kê mô tả bốn thuộc tính của 59 khách (kết quả df2.describe() trong CustomerCluster1.py).

4.3. Phân phối của các thuộc tính

AvgMinutes lệch phải rõ rệt: trung vị 5,16 phút nhưng giá trị lớn nhất 16,55 phút. Đuôi dài này do một nhóm nhỏ khách mua nhiều nội dung video tạo ra; chính nhóm này sẽ tách thành một cụm riêng ở Chương 6. GenreCount và AlbumCount nhận giá trị nguyên nên histogram có dạng cột rời. AlbumPopularity tập trung quanh 5,2 với độ lệch chuẩn 0,53, chỉ 6/59 khách có giá trị dưới 4,5.



Hình 4.3. Histogram GenreCount và AvgMinutes (hàm showHistogram, CustomerCluster1.py).



Hình 4.4. Histogram AlbumCount và AlbumPopularity (CustomerCluster2.py).

4.4. Chuẩn hóa Min-Max

K-Means gán mỗi khách vào cụm có tâm gần nhất theo khoảng cách Euclid. Nếu để nguyên đơn vị gốc, thuộc tính có khoảng giá trị rộng sẽ chi phối khoảng cách. AvgMinutes trải rộng 13 đơn vị so với 8 của GenreCount. AlbumCount trải rộng 9 đơn vị so với chưa tới 3 của AlbumPopularity. Khi đó AlbumPopularity gần như không ảnh hưởng đến kết quả gom cụm. Chuẩn hóa Min-Max đưa mọi thuộc tính về đoạn [0, 1]:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Với khách 12: GenreCount' = (5 − 4) / (12 − 4) = 0,125 và AvgMinutes' = (3,82 − 3,56) / (16,55 − 3,56) = 0,020. Hàm minMaxScale trả về cả min và max của từng cột. Hai giá trị này được lưu cùng mô hình để chuẩn hóa khách mới theo đúng thang đo lúc huấn luyện (Chương 7). Biểu đồ phân tán vẫn vẽ bằng giá trị gốc để người đọc nhìn trục là hiểu ngay "9 thể loại" hay "12 phút".

CHƯƠNG 5. GIẢI THUẬT K-MEANS VÀ LỰA CHỌN SỐ CỤM

Bài làm dùng K-Means của scikit-learn với các tham số theo bài giảng 07. Số cụm K được chọn bằng phương pháp Elbow, trong đó điểm khuỷu tay được xác định tự động bằng thuật toán Kneedle và được kiểm tra lại bằng hệ số Silhouette. Kết quả là K = 3 cho thực nghiệm track, K = 3 cho thực nghiệm album và K = 4 cho thực nghiệm ba thuộc tính. Bài toán gom cụm không có nhãn đúng, nên không chia tập train/test như bài hồi quy; toàn bộ 59 khách đều được dùng để huấn luyện.

5.1. Cơ chế hoạt động

K-Means tìm K tâm cụm sao cho tổng bình phương khoảng cách từ mỗi điểm tới tâm cụm của nó là nhỏ nhất. Tổng này gọi là WCSS (within-cluster sum of squares), trong scikit-learn là thuộc tính inertia_:

$$WCSS = \sum_{j=1}^{K} \sum_{x_i \in C_j} \|x_i - c_j\|^2$$

Thuật toán lặp bốn bước giống ví dụ tính tay trong KMeans_Bai1.xlsx:

1. Khởi tạo K tâm cụm.
2. Tính khoảng cách Euclid từ mỗi khách tới từng tâm, gán khách vào cụm có tâm gần nhất.
3. Cập nhật tâm mỗi cụm bằng trung bình cộng các khách trong cụm.
4. Nếu tâm không đổi (hoặc đạt số vòng lặp tối đa) thì dừng, ngược lại quay lại bước 2.

5.2. Tham số mô hình

Tham số	Giá trị	Lý do
n_clusters	3 hoặc 4	Chọn bằng Elbow + Kneedle, kiểm tra bằng Silhouette (mục 5.3–5.4)
init	'k-means++'	Chọn tâm ban đầu cách xa nhau, tránh hai tâm rơi vào cùng một vùng dữ liệu
max_iter	500	Giống bài giảng; với 59 điểm, thuật toán hội tụ trước giới hạn này
n_init	10	Chạy 10 lần với 10 bộ tâm ban đầu khác nhau, giữ lần có WCSS nhỏ nhất; ghi rõ để kết quả không phụ thuộc phiên bản scikit-learn
random_state	42	Mỗi lần chạy cho cùng một kết quả

```python
def findElbow(X, maxK=10, tolerance=0.9):  2 usages
    kRange = np.arange(1, maxK + 1)
    inertia = []
    for n in kRange:
        model = KMeans(n_clusters=n,
                       init='k-means++',
                       max_iter=500,
                       n_init=10,
                       random_state=42)
        model.fit(X)
        inertia.append(model.inertia_)
    inertia = np.array(inertia)
    xNorm = (kRange - kRange.min()) / (kRange.max() - kRange.min())
    yNorm = (inertia - inertia.min()) / (inertia.max() - inertia.min() + 1e-12)
    distance = (1 - xNorm) - yNorm
    optimalK = int(kRange[distance.argmax()])
    elbowZone = kRange[distance >= tolerance * distance.max()]
    return kRange, inertia, elbowZone, optimalK

def elbowMethod(df, columnsForElbow, maxK=10, tolerance=0.9):  3 usages
    X = df.loc[:, columnsForElbow].values
    kRange, inertia, elbowZone, optimalK = findElbow(X, maxK, tolerance)

    print("Inertia theo K:", dict(zip(kRange.tolist(), np.round(inertia, decimals: 2).tolist())))
    print("Khoảng elbow:", elbowZone.tolist(), "=> điểm gập lớn nhất K =", optimalK)

    plt.figure(figsize=(15, 6))
    plt.plot( *args: kRange, inertia, 'o')
    plt.plot( *args: kRange, inertia, '-', alpha=0.5)
    if len(elbowZone) > 1:
        plt.axvspan(elbowZone.min() - 0.3, elbowZone.max() + 0.3, color='orange', alpha=0.15,
                    label=f'Khoảng elbow: K = {elbowZone.min()} - {elbowZone.max()}')
    plt.axvline(x=optimalK, color='red', linestyle='--', linewidth=1.5,
                label=f'Điểm gập lớn nhất K = {optimalK}')
    plt.plot( *args: optimalK, inertia[optimalK - 1], 'o', color='red', markersize=12)
    plt.xticks(kRange)
    plt.xlabel('Number of Clusters'), plt.ylabel('Cluster sum of squared distances')
    plt.title('Elbow Method (' + ' X '.join(columnsForElbow) + ')')
    plt.legend()
    plt.show()
    return optimalK
```

Hình 5.1. Hàm findElbow (thuật toán Kneedle) và elbowMethod

5.3. Chọn số cụm K

Phương pháp Elbow chạy K-Means với K từ 1 đến 10 và vẽ WCSS theo K. WCSS luôn giảm khi K tăng. Điểm cần tìm là "khuỷu tay": từ đó trở đi, thêm một cụm chỉ làm WCSS giảm ít. Vì đồ thị Elbow đôi khi không có khuỷu rõ, bài làm tính thêm hệ số Silhouette. Với khách i, a(i) là khoảng cách trung bình tới các khách cùng cụm, b(i) là khoảng cách trung bình tới các khách của cụm gần nhất còn lại:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i),\ b(i)\}}$$

Silhouette của mô hình là trung bình s(i) trên 59 khách, nằm trong [−1, 1]. Giá trị càng gần 1 thì cụm càng tách biệt. Bảng dưới đây tóm tắt các giá trị dùng để ra quyết định; dòng in đậm là K được chọn.

Thực nghiệm	**K**	**WCSS**	**Mức giảm WCSS so với K − 1**	**Silhouette**
1. GenreCount × AvgMinutes	2	2,89	−47%	0,492
	3	**1,74**	**−40%**	**0,434**
	4	1,33	−24%	0,399
2. AlbumCount × AlbumPopularity	2	2,58	−54%	0,439
	3	**1,80**	**−30%**	**0,414**
	4	1,35	−25%	0,383
3. GenreCount × AvgMinutes × AlbumCount	3	3,88	−31%	0,315
	4	**3,01**	**−22%**	**0,313**
	5	2,49	−17%	0,316

Thực nghiệm 1. WCSS giảm 40% khi đi từ K = 2 lên K = 3, nhưng chỉ giảm 24% khi lên K = 4. Khuỷu tay nằm ở K = 3. K = 2 có Silhouette cao hơn (0,492) nhưng chỉ tách theo thời lượng: 45 khách mua track ngắn và 14 khách có thời lượng trung bình cao. Khi đó, khách nghe 8–10 thể loại nằm chung cụm với khách chỉ nghe 4–6 thể loại, và GenreCount gần như không có vai trò. K = 3 tách được cả hai chiều (Chương 6) mà Silhouette vẫn ở mức 0,434, nên được chọn.

Thực nghiệm 2. Mức giảm WCSS lần lượt là 54%, 30% và 25%. Sau K = 3, mỗi cụm thêm vào chỉ làm WCSS giảm khoảng một phần tư, nên chọn K = 3 với Silhouette 0,414.

Thực nghiệm 3. Elbow không có khuỷu rõ; Silhouette từ K = 3 đến K = 5 gần như bằng nhau (0,313–0,316). Khi chỉ số không phân định được, K được chọn theo khả năng diễn giải. Với K = 3, ngoài nhóm mua video, chương trình chỉ chia các khách còn lại theo số album: 28 khách nhiều album và 20 khách ít album. Trong mỗi nhóm, khách nghe 9–10 thể loại vẫn nằm lẫn với khách nghe 5–6 thể loại. K = 4 tách 15 khách nghe đa dạng thể loại (8–10) thành cụm riêng, nên mỗi cụm đặt tên được rõ ràng (Chương 6).

5.4. Tự động chọn K bằng thuật toán Kneedle

Đọc khuỷu tay bằng mắt như mục 5.3 phụ thuộc vào người xem: cùng một đồ thị, người này thấy khuỷu ở K = 3, người khác thấy ở K = 4. Thuật toán Kneedle (Satopää và cộng sự, 2011) biến việc này thành một phép tính. Hàm findElbow (Hình 5.1) cài đặt thuật toán qua ba bước:

1. Chuẩn hóa K và WCSS về đoạn [0, 1]. Sau bước này, đường cong đi từ điểm (0, 1) xuống điểm (1, 0), và đường thẳng nối hai đầu có phương trình y = 1 − x.
2. Với mỗi K, tính độ gập d = (1 − x) − y, tức khoảng cách theo chiều dọc từ đường thẳng xuống đường cong. Khuỷu tay là chỗ đường cong võng xa đường thẳng nhất, nên K có d lớn nhất là điểm khuỷu tay.
3. Các K có d ≥ 0,9 × d lớn nhất tạo thành khoảng elbow, tức những K có độ gập gần ngang với điểm tốt nhất. Ngưỡng 0,9 là tham số tolerance của hàm.

$$d_K = (1 - x_K) - y_K, \quad x_K = \frac{K-1}{K_{max}-1}, \quad y_K = \frac{WCSS_K - WCSS_{min}}{WCSS_{max} - WCSS_{min}}$$

Ví dụ với thực nghiệm 1: K = 3 có WCSS = 1,744, nên x = 2/9 = 0,222 và y = (1,744 − 0,311) / (5,431 − 0,311) = 0,280. Độ gập d = (1 − 0,222) − 0,280 = 0,498, lớn nhất trong mười giá trị K. K = 4 có d = 0,467, bằng 94% giá trị lớn nhất nên cũng thuộc khoảng elbow. K = 5 có d = 0,427, chỉ bằng 86%, nên bị loại.

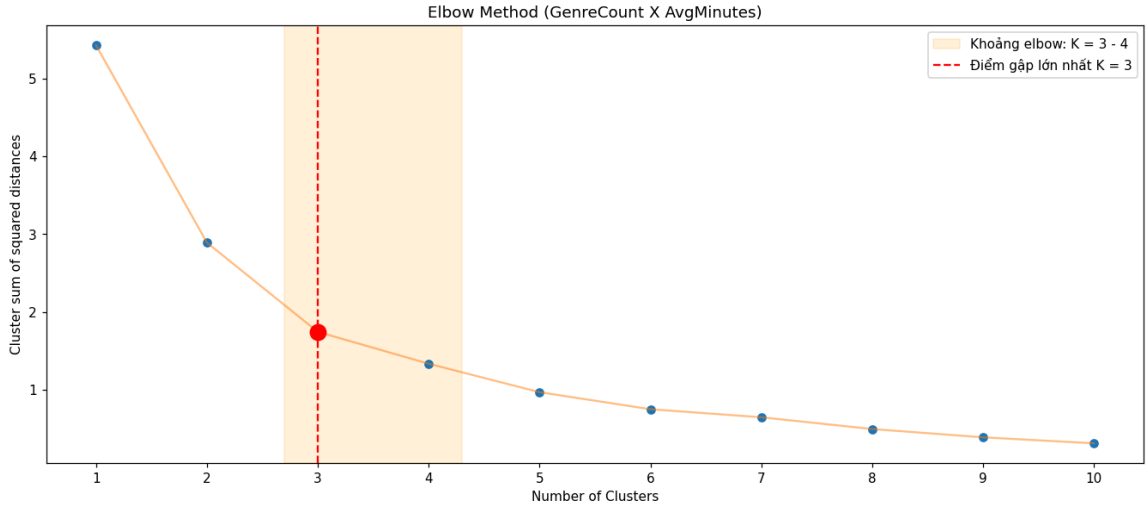
Thực nghiệm	d tại K = 2 / 3 / 4 / 5	Khoảng elbow	K tự chọn	Silhouette
1. GenreCount × AvgMinutes	0,385 / **0,498** / 0,467 / 0,427	3 – 4	3	0,434
2. AlbumCount × AlbumPopularity	0,479 / **0,520** / 0,496 / 0,457	2 – 4	3	0,414
3. GenreCount × AvgMinutes × AlbumCount	0,307 / 0,428 / **0,429** / 0,386	3 – 4	4	0,313

Cả ba thực nghiệm, Kneedle cho cùng K với cách đọc đồ thị ở mục 5.3, nên hai cách kiểm tra chéo cho nhau. Sau khi có Kneedle, hàm elbowMethod trả K về cho file thực nghiệm (cluster = elbowMethod(dfScaled, columns)), nên số cụm không còn phải gõ tay.

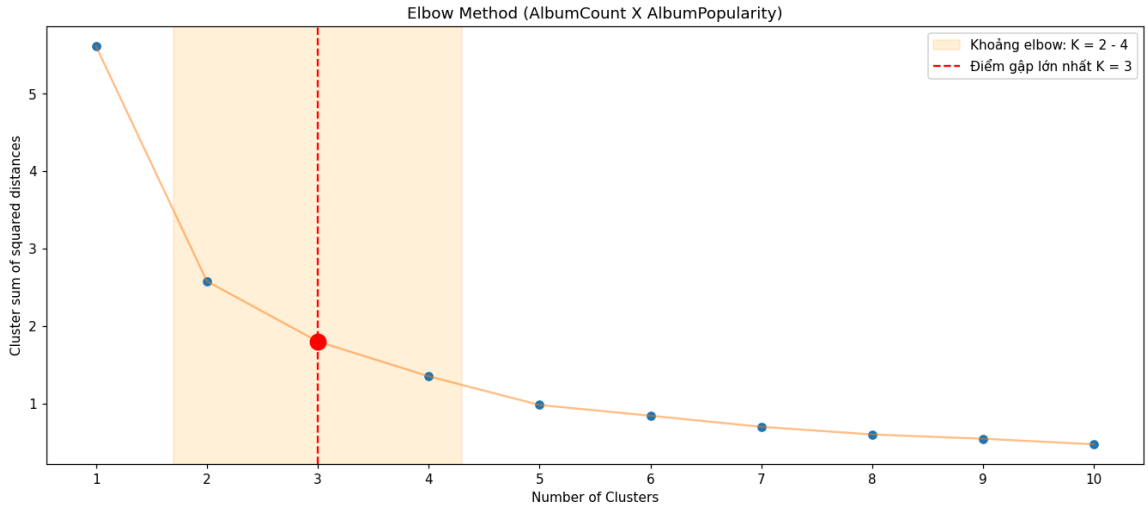
Bài học trên lớp gợi ý: khi khoảng elbow có nhiều giá trị thì ưu tiên K lớn hơn để phân khúc chi tiết hơn. Bài làm không theo quy tắc này mà lấy K có độ gập lớn nhất. Nếu lấy K lớn nhất trong khoảng elbow, thực nghiệm 1 phải dùng K = 4 và Silhouette giảm từ 0,434 xuống 0,399; thực nghiệm 2 cũng phải dùng K = 4 và Silhouette giảm từ 0,414 xuống 0,383. So nhãn cụm giữa K = 3 và K = 4 ở thực nghiệm 1 cho thấy cụm thứ tư

không phải một nhóm hành vi mới: nó gồm 25 khách cắt ra từ cả ba cụm cũ (15 khách của cụm 3, 8 khách của cụm 1 và 2 khách của cụm 2). Ranh giới mới đi xuyên qua vùng dữ liệu dày, nên các cụm kém tách biệt hơn mà không thêm được phân khúc nào dễ diễn giải.

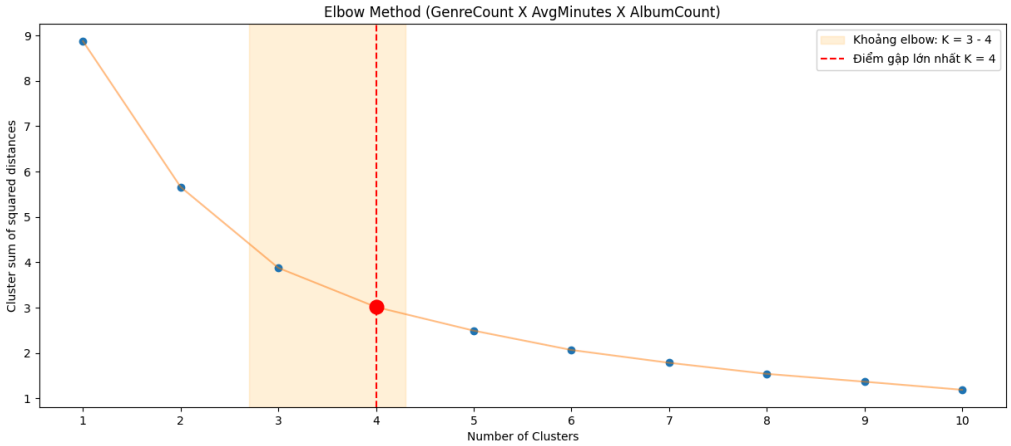
Ở thực nghiệm 3, độ gập tại K = 3 và K = 4 chênh nhau chưa tới 0,002, nên về mặt thuật toán hai giá trị gần như ngang nhau. Kneedle chọn K = 4, trùng với lập luận diễn giải ở mục 5.3. K = 5 có d = 0,3859, sát ngưỡng 0,3863 nhưng vẫn nằm ngoài khoảng elbow. Ba đồ thị dưới đây là kết quả của elbowMethod: vùng màu cam là khoảng elbow, đường đỏ là K được chọn.



Hình 5.2. Đồ thị Elbow của thực nghiệm 1 (GenreCount × AvgMinutes): khoảng elbow K = 3–4, điểm gập lớn nhất K = 3.



Hình 5.3. Đồ thị Elbow của thực nghiệm 2 (AlbumCount × AlbumPopularity): khoảng elbow K = 2–4, điểm gập lớn nhất K = 3.



Hình 5.4. Đồ thị Elbow của thực nghiệm 3 (ba thuộc tính): khoảng elbow K = 3–4, điểm gập lớn nhất K = 4.

CHƯƠNG 6. KẾT QUẢ HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH

Cả ba thực nghiệm đều tạo ra các cụm diễn giải được. Silhouette đạt 0,434 (track), 0,414 (album) và 0,313 (ba thuộc tính). Hai mô hình hai thuộc tính tách cụm tốt hơn mô hình ba thuộc tính. Mô hình ba thuộc tính đổi lại cho phân khúc chi tiết hơn. Mỗi thực nghiệm chạy cùng một trình tự trong file CustomerCluster*.py: tạo thuộc tính → histogram → chuẩn hóa → Elbow + Kneedle (trả về K) → runKMeans → in WCSS và Silhouette → vẽ cụm → thống kê cụm → danh sách khách theo cụm → xuất Excel → lưu mô hình.

```
conn = getConnect(DATABASE_PATH)
df2 = getCustomerFeatures(conn)
print(df2)
print(df2.head())
print(df2.describe())
columns = ["GenreCount", "AvgMinutes"]
showHistogram(df2, columns)
# Chuẩn hóa Min-Max trước khi gom cụm
dfScaled, minValues, maxValues = minMaxScale(df2, columns)
print(dfScaled.head())
# Elbow + Kneedle tự tìm số cụm (điểm gập lớn nhất)
cluster = elbowMethod(dfScaled, columns)
print("Số cụm được chọn: K =", cluster)
X = dfScaled.loc[:, columns].values
colors = ["red", "green", "blue", "purple", "black", "pink", "orange"]
y_kmeans, centroids, labels, model = runKMeans(X, cluster)
print(y_kmeans)
print(centroids)
print(labels)
df2["cluster"] = labels
# Đánh giá chất lượng gom cụm: Silhouette càng gần 1 càng tốt, WCSS (inertia) càng nhỏ càng chặt
print("WCSS (inertia):", round(model.inertia_, 4))
print("Silhouette    :", round(silhouette_score(X, labels), 4))
# Trực quan hóa bằng giá trị gốc (chưa chuẩn hóa) để dễ đọc trục
visualizeKMeans(df2.loc[:, columns].values, y_kmeans,
                cluster,
                title: "Clusters of Customers - GenreCount X AvgMinutes",
                xlabel: "GenreCount (số thể loại đã mua)",
                ylabel: "AvgMinutes (thời lượng trung bình track - phút)",
                colors)
showClusterSummary(df2, columns)
showCustomerDetailByCluster(conn, df2, cluster)
# Xuất Excel: mỗi cụm 1 sheet + sheet Thống kê
dfDetail = getClusterDetail(conn, df2, columns)
exportClustersToExcel(dfDetail, columns, os.path.join(EXCEL_DIR, "CustomerCluster1_Track.xlsx"))
# Lưu mô hình và thông tin chuẩn hóa để dùng lại trong using_model.py, export_results.py, giao diện
modelFile, scalerFile = getModelPaths("kmeans_track")
FileUtil.savemodel(model, modelFile)
FileUtil.savemodel( model: {"columns": columns, "min": minValues, "max": maxValues}, scalerFile)
print("Đã lưu mô hình:", modelFile)
closeConnection(conn)
```

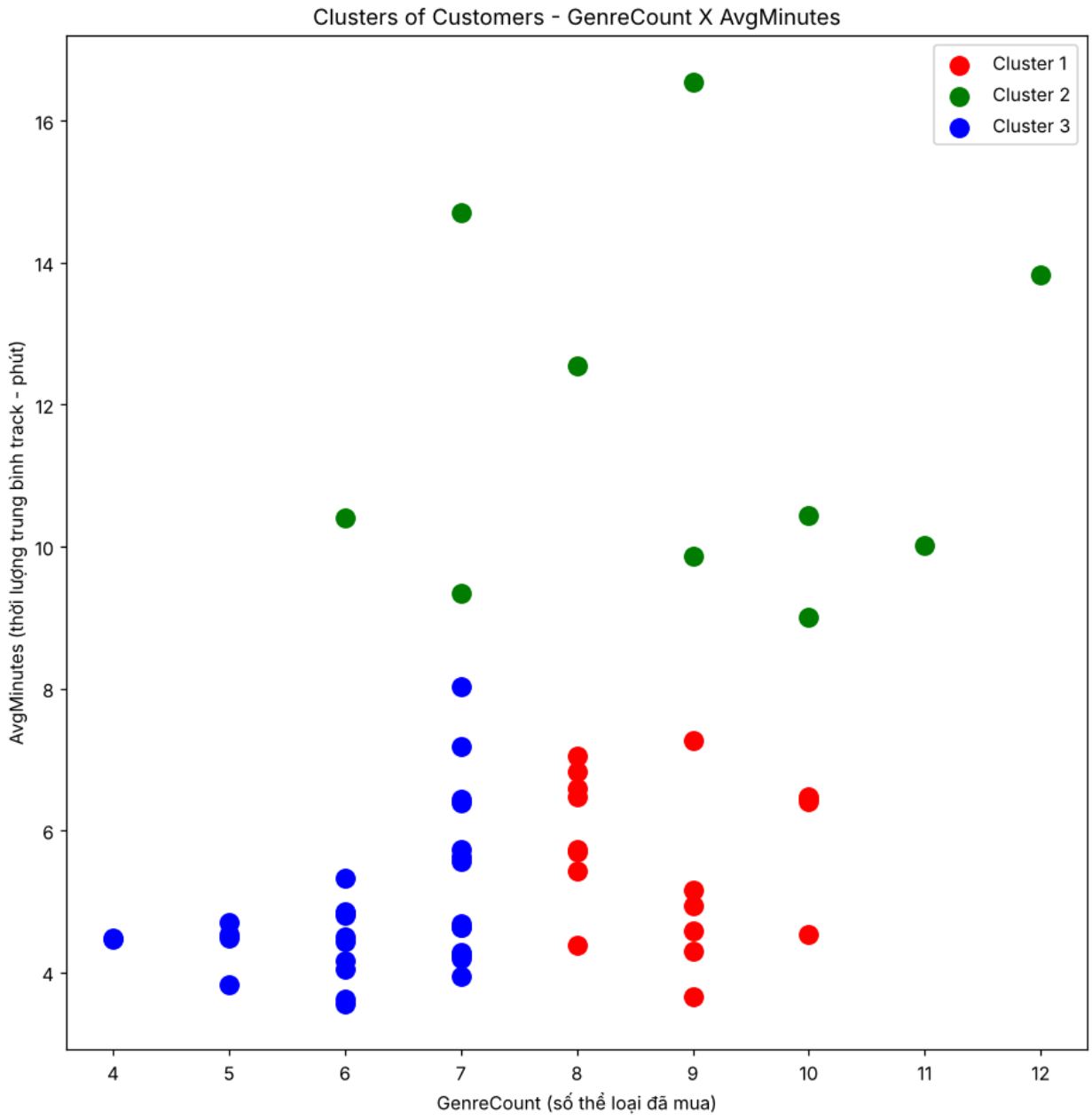
Hình 6.1. Toàn bộ file Clustering/CustomerCluster1.py. Hai file CustomerCluster2.py và CustomerCluster3.py có cùng cấu trúc, chỉ khác danh sách cột, tên file Excel và tên file mô hình; số cụm do elbowMethod trả về.

6.1. Thực nghiệm 1: quan tâm track (GenreCount × AvgMinutes, K = 3)

WCSS = 1,7442 và Silhouette = 0,4338. Ba cụm tách nhau theo hai chiều: trục ngang (số thể loại) tách cụm 3 khỏi cụm 1, trục dọc (thời lượng) tách cụm 2 khỏi hai cụm còn lại.

```
Inertia theo K: {1: 5.43, 2: 2.89, 3: 1.74, 4: 1.33, 5: 0.97, 6: 0.75, 7: 0.65, 8: 0.49, 9: 0.39, 10: 0.31}
Khoảng elbow: [3, 4] => điểm gập lớn nhất K = 3
Số cụm được chọn: K = 3
[0 2 0 0 0 1 1 2 2 2 2 2 2 0 0 2 0 2 0 2 0 0 0 1 0 1 0 1 2 2 2 2 2 0 2 2 1
 2 2 2 0 2 0 1 1 1 2 2 2 2 2 2 2 0 2 2 1 0 2]
[[0.59722222 0.16204773]
 [0.6125     0.62448037]
 [0.27822581 0.09960516]]
[0 2 0 0 0 1 1 2 2 2 2 2 2 0 0 2 0 2 0 2 0 0 0 1 0 1 0 1 2 2 2 2 2 0 2 2 1
 2 2 2 0 2 0 1 1 1 2 2 2 2 2 2 2 0 2 2 1 0 2]
WCSS (inertia): 1.7442
Silhouette    : 0.4338
         GenreCount  AvgMinutes  NumberOfCustomers
cluster
1              8.78        5.66                 18
2              8.90       11.67                 10
3              6.23        4.85                 31
```

Hình 6.2. Nhãn cụm, tâm cụm (đã chuẩn hóa), WCSS, Silhouette và bảng thống kê cụm của thực nghiệm 1.



Hình 6.3. Phân bố 59 khách theo GenreCount và AvgMinutes, tô màu theo cụm.

Cụm	**Số khách**	**GenreCount (TB, khoảng)**	**AvgMinutes (TB, khoảng)**	**Lượt mua là video**	**Chi tiêu TB**	**Tên cụm**
1	18	8,78 (8–10)	5,66 (3,66–7,27)	3,8%	39,06 USD	Nghe đa dạng thể loại

Cụm	Số khách	GenreCount (TB, khoảng)	AvgMinutes (TB, khoảng)	Lượt mua là video	Chi tiêu TB	Tên cụm
2	10	8,90 (6–12)	11,67 (9,01–16,55)	19,5%	45,02 USD	Khách mua nội dung video
3	31	6,23 (4–7)	4,85 (3,56–8,03)	0,9%	37,91 USD	Tập trung ít thể loại

Cụm 3 là nhóm lớn nhất (31 khách, 53%). Nhóm này chỉ nghe 4–7 thể loại, Rock chiếm 41,7% lượt mua, cao nhất trong ba cụm. Cụm 1 có cùng mức thời lượng với cụm 3 nhưng nghe 8–10 thể loại, và Rock chỉ chiếm 31,6%. Cụm 2 nhỏ nhất nhưng đáng chú ý nhất: gần một phần năm lượt mua là nội dung video giá 1,99 USD, nên chi tiêu trung bình cao hơn hai cụm còn lại khoảng 6–7 USD mỗi khách.

```
Cluster 2: có 10 customer
 CustomerId FirstName   LastName Company           City        Country                    Email  cluster
          6    Helena       Holý     NaN         Prague Czech Republic          hholy@gmail.com        2
          7    Astrid     Gruber     NaN         Vienne        Austria    astrid.gruber@apple.at        2
         24     Frank    Ralston     NaN        Chicago            USA        fralston@gmail.com        2
         26   Richard Cunningham     NaN     Fort Worth            USA  ricunningham@hotmail.com        2
         28     Julia    Barnett     NaN Salt Lake City            USA       jubarnett@gmail.com        2
         37      Fynn Zimmermann     NaN      Frankfurt        Germany      fzimmermann@yahoo.de        2
         44     Terhi Hämäläinen     NaN       Helsinki        Finland terhi.hamalainen@apple.fi        2
         45  Ladislav     Kovács     NaN       Budapest        Hungary  ladislav_kovacs@apple.hu        2
         46      Hugh   O'Reilly     NaN         Dublin        Ireland      hughoreilly@apple.ie        2
         57      Luis      Rojas     NaN       Santiago          Chile        luisrojas@yahoo.cl        2
```

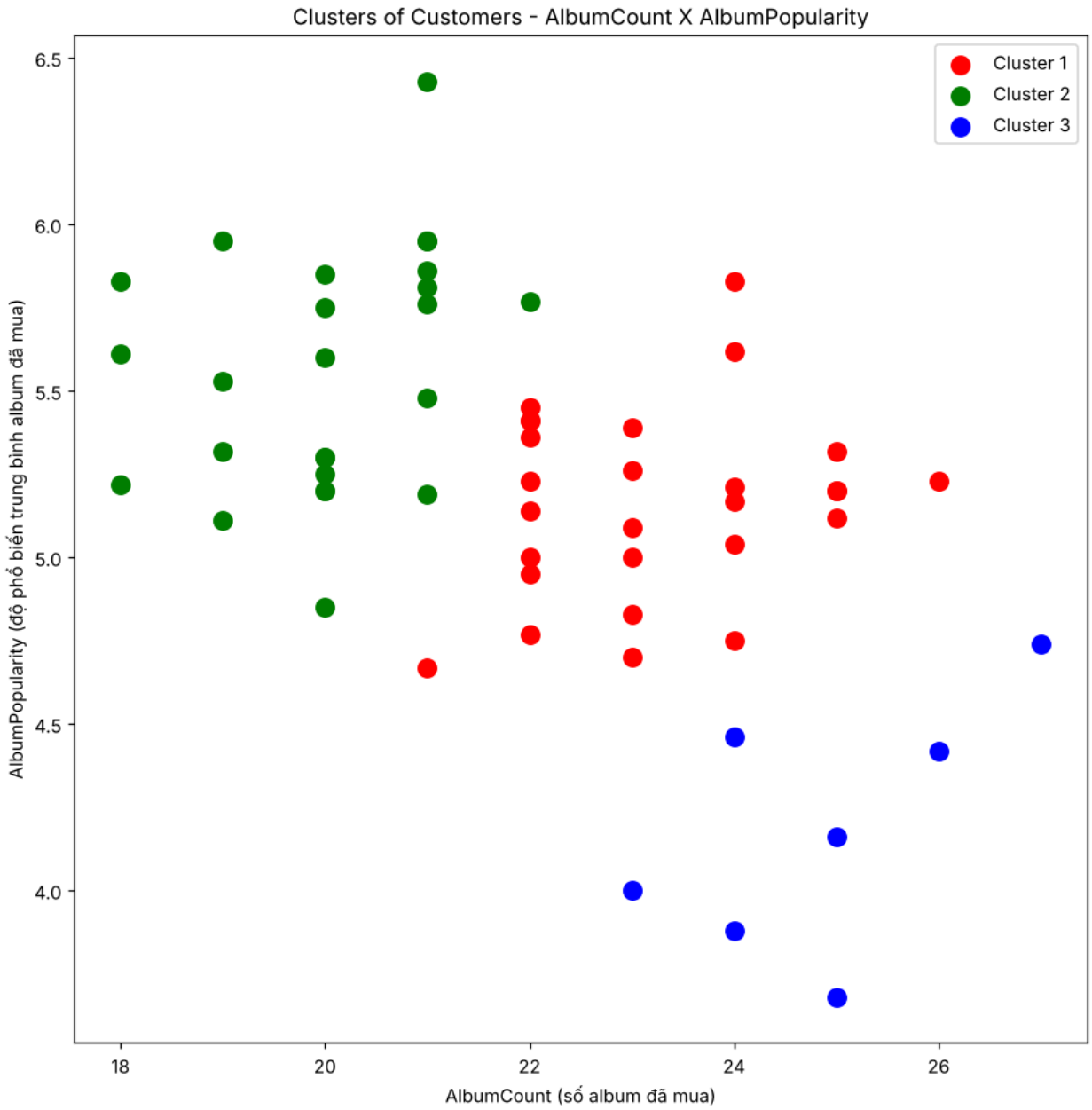
Hình 6.4. Danh sách chi tiết 10 khách thuộc cụm 2, in bởi hàm showCustomerDetailByCluster (Bài tập 1, bài giảng 07).

6.2. Thực nghiệm 2: quan tâm album (AlbumCount × AlbumPopularity, K = 3)

WCSS = 1,799 và Silhouette = 0,4135. Hai thuộc tính tương quan nghịch (hệ số −0,49): khách mua càng nhiều album thì càng hay chạm tới album ít người mua. Vì vậy các cụm xếp thành dải chéo từ góc trên bên trái xuống góc dưới bên phải của biểu đồ.

```
Inertia theo K: {1: 5.62, 2: 2.58, 3: 1.8, 4: 1.35, 5: 0.98, 6: 0.84, 7: 0.7, 8: 0.6, 9: 0.54, 10: 0.47}
Khoảng elbow: WCSS (inertia): 1.799
Silhouette    : 0.4135
         AlbumCount  AlbumPopularity  NumberOfCustomers
cluster
1             23.22             5.16                 27
2             20.00             5.56                 25
3             24.86             4.19                  7
```

Hình 6.5. WCSS, Silhouette và bảng thống kê cụm của thực nghiệm 2.



Hình 6.6. Phân bố 59 khách theo AlbumCount và AlbumPopularity.

Cụm	**Số khách**	**AlbumCount (TB, khoảng)**	**AlbumPopularity (TB, khoảng)**	**Album được nhiều khách trong cụm mua nhất**	**Tên cụm**
1	27	23,22 (21–26)	5,16 (4,67–5,83)	Unplugged – Eric Clapton (9 khách)	Khám phá rộng, album phổ thông

Cụm	Số khách	AlbumCount (TB, khoảng)	AlbumPopularity (TB, khoảng)	Album được nhiều khách trong cụm mua nhất	Tên cụm
2	25	20,00 (18–22)	5,56 (4,85–6,43)	Greatest Hits – Lenny Kravitz (10 khách)	Theo album được nhiều người mua
3	7	24,86 (23–27)	4,19 (3,68–4,74)	Led Zeppelin III – Led Zeppelin (4 khách)	Săn album ít người biết

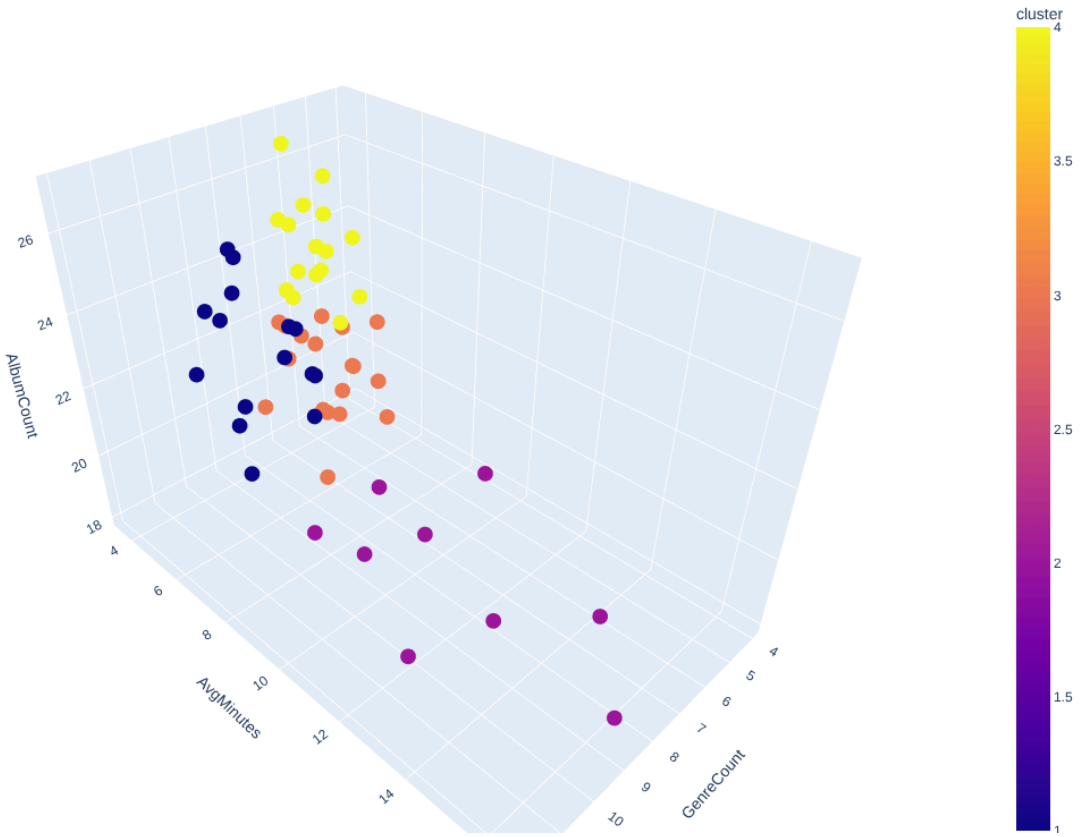
Cụm 2 mua ít album nhất (18–22) nhưng chọn các album được nhiều khách khác cùng mua. 10/25 khách của cụm có mua *Greatest Hits* của Lenny Kravitz. Cụm 3 chỉ có 7 khách nhưng có hành vi trái ngược: mua từ 23 đến 27 album khác nhau, độ phổ biến trung bình thấp nhất (4,19). Thể loại Classical chiếm 9,4% lượt mua của cụm này, trong khi Classical chỉ chiếm 1,8% tổng lượt mua toàn cửa hàng.

6.3. Thực nghiệm 3: kết hợp track và album (ba thuộc tính, K = 4)

WCSS = 3,0133 và Silhouette = 0,3129. Silhouette thấp hơn hai thực nghiệm trước là điều dự kiến: thêm một chiều thì khoảng cách giữa các cụm bị "pha loãng" hơn. Bù lại, mô hình chia nhóm nghe ít thể loại thành hai kiểu khác nhau về cách mua album, điều mà thực nghiệm 1 không thấy được.

```
Inertia theo K: {1: 8.87, 2: 5.66, 3: 3.88, 4: 3.01, 5: 2.49, 6: 2.07, 7: 1.78, 8: 1.54, 9: 1.37, 10: 1.19}
Khoảng elbow: WCSS (inertia): 3.0133
Silhouette    : 0.3129
[plotly figure saved]
         GenreCount  AvgMinutes  AlbumCount  NumberOfCustomers
cluster
1              9.07        5.80       22.73                 15
2              8.78       11.97       19.67                  9
3              6.26        4.89       20.63                 19
4              6.62        5.10       24.44                 16
```

Hình 6.7. WCSS, Silhouette và bảng thống kê cụm của thực nghiệm 3.



Hình 6.8. Biểu đồ 3D của thực nghiệm 3, vẽ bằng hàm visualize3DKmeans (plotly) và mở trên trình duyệt.

Cụm	Số khách	GenreCount TB	AvgMinutes TB	AlbumCount TB	Tên cụm
1	15	9,07	5,80	22,73	Nghe đa dạng thể loại
2	9	8,78	11,97	19,67	Khách mua nội dung video
3	19	6,26	4,89	20,63	Trung thành, ít thể loại và ít album
4	16	6,62	5,10	24,44	Ít thể loại nhưng nhiều album

Cụm 1 và cụm 2 gần như trùng với cụm 1 và cụm 2 của thực nghiệm 1 (15 so với 18 khách, 9 so với 10 khách). Điều này cho thấy hai nhóm "nghe đa dạng" và "mua video" ổn định khi thêm thuộc tính mới. Khác biệt nằm ở 31 khách nghe ít thể loại: nay họ tách thành cụm 3 (19 khách, trung bình 20,6 album) và cụm 4 (16 khách, trung bình 24,4

album). Cụm 4 nghe ít thể loại nhưng trải rộng trên nhiều album, Rock chiếm 42,3% và Metal 15,3%. Đây là nhóm phù hợp nhất với chiến dịch gợi ý album mới cùng thể loại.

6.4. Đánh giá chung

Thực nghiệm	Thuộc tính	K	WCSS	Silhouette	Nhận xét
1	GenreCount, AvgMinutes	3	1,7442	0,4338	Tách cụm tốt nhất, dễ diễn giải
2	AlbumCount, AlbumPopularity	3	1,7990	0,4135	Tách cụm tốt, phục vụ gợi ý album
3	GenreCount, AvgMinutes, AlbumCount	4	3,0133	0,3129	Tách cụm yếu hơn, phân khúc chi tiết hơn

WCSS giữa các thực nghiệm không so sánh trực tiếp được, vì số chiều và số cụm khác nhau. Silhouette mới là thước đo chung. Theo thang tham khảo thường dùng, Silhouette từ 0,26 đến 0,50 cho thấy cấu trúc cụm có tồn tại nhưng ranh giới giữa các cụm chưa sắc nét. Cả ba mô hình đều nằm trong khoảng này. Một phần nguyên nhân là Chinook là dữ liệu mẫu, hành vi mua được sinh khá đều giữa các khách (Chương 1). Một phần khác là các cụm nằm sát nhau trên thang đo rời rạc của GenreCount và AlbumCount (Chương 9).

Khi xuất kết quả ra Excel và HTML (Chương 7), mỗi cụm cần một cái tên thay cho con số. Hàm nameClusters (Hình 6.9) đặt tên tự động theo z-score: với mỗi thuộc tính, z = (trung bình cụm − trung bình 59 khách) / độ lệch chuẩn. Thuộc tính có |z| lớn nhất quyết định tên chính; thuộc tính thứ hai được ghép thêm khi |z| ≥ 0,5. Chữ dùng cho mỗi chiều lệch lấy từ từ điển FEATURE_WORDS, ví dụ AvgMinutes cao là "Mua nội dung video dài", thấp là "Nghe track ngắn".

Với thực nghiệm 1, trung bình toàn bộ là GenreCount = 7,46 (độ lệch chuẩn 1,71) và AvgMinutes = 6,26 (2,85). Cụm 2 có AvgMinutes = 11,67, z = 1,90, và GenreCount = 8,90, z = 0,85, nên được đặt tên "Mua nội dung video dài · Nghe đa dạng thể loại". Cụm 3 có z(GenreCount) = −0,72 và z(AvgMinutes) = −0,49; thuộc tính thứ hai chưa tới ngưỡng 0,5 nên tên chỉ gồm một vế "Tập trung ít thể loại". Tên tự động của cả ba thực nghiệm khớp về ý nghĩa với tên đặt sau khi đọc số liệu ở các bảng ở mục 6.1–6.3; báo cáo giữ tên đọc số liệu vì chúng nói rõ hơn, còn file Excel và trang HTML dùng tên tự động.

```python
# 7. Đặt tên cụm tự động: so sánh trung bình của cụm với trung bình toàn bộ khách theo z-score,
#    lấy thuộc tính lệch nhiều nhất (và thuộc tính thứ hai nếu lệch >= 0.5 độ lệch chuẩn)
FEATURE_WORDS = {
    "GenreCount": ("Nghe đa dạng thể loại", "Tập trung ít thể loại"),
    "AvgMinutes": ("Mua nội dung video dài", "Nghe track ngắn"),
    "AlbumCount": ("Mua rải rác nhiều album", "Gom vào ít album"),
    "AlbumPopularity": ("Theo album phổ biến", "Săn album ít người biết"),
}


def nameClusters(df, columns, labelColumn="cluster"):  1 usage
    mean = df[columns].mean()
    std = df[columns].std()
    names = {}
    for c in sorted(df[labelColumn].unique()):
        z = (df[df[labelColumn] == c][columns].mean() - mean) / std
        order = z.abs().sort_values(ascending=False).index
        words = []
        for col in order[:2]:
            if len(words) == 0 or abs(z[col]) >= 0.5:
                words.append(FEATURE_WORDS[col][0] if z[col] > 0 else FEATURE_WORDS[col][1])
        names[c] = " · ".join(words)
    return names
```

Hình 6.9. Từ điển FEATURE_WORDS và hàm nameClusters

CHƯƠNG 7. LƯU VÀ SỬ DỤNG MÔ HÌNH

Mỗi thực nghiệm lưu hai file vào Trained/: file mô hình K-Means và file thông tin chuẩn hóa. Mô hình đã lưu được dùng lại ở ba nơi mà không cần huấn luyện lại: using_model.py trên console (mục 7.2–7.3), export_results.py tạo trang HTML (mục 7.4) và giao diện PyQt6 (Chương 8). Trên console, chương trình thực hiện ba việc: xác định cụm của một khách có sẵn, gợi ý album cho khách đó và dự đoán cụm cho một khách mới. Đây là bước (9)–(11) trong sơ đồ hệ thống máy học ở bài giảng 08.

7.1. Nội dung được lưu

File	Nội dung	Kích thước
kmeans_track_model.zip	Đối tượng KMeans của thực nghiệm 1 (kèm cluster_centers_)	882 byte
kmeans_track_scaler.zip	Từ điển {"columns", "min", "max"} dùng để chuẩn hóa	1.174 byte
kmeans_album_model.zip, kmeans_album_scaler.zip	Tương tự cho thực nghiệm 2	882 / 1.179 byte
kmeans_track_album_model.zip, kmeans_track_album_scaler.zip	Tương tự cho thực nghiệm 3	930 / 1.211 byte

Lưu riêng min và max là bắt buộc. Mô hình được huấn luyện trên dữ liệu đã chuẩn hóa, nên một khách mới phải được chuẩn hóa bằng đúng min, max của lần huấn luyện. Nếu tính lại min, max từ dữ liệu mới, cùng một giá trị GenreCount = 5 có thể được quy đổi thành một con số khác, và khách sẽ bị xếp sai cụm. Việc lưu và tải dùng lại lớp FileUtil của bài Housing price prediction (pickle.dump và pickle.load).

```python
# 1. Tải mô hình và thông tin chuẩn hóa
trackModelFile, trackScalerFile = getModelPaths("kmeans_track")
albumModelFile, albumScalerFile = getModelPaths("kmeans_album")
trackModel = FileUtil.loadmodel(trackModelFile)
trackScaler = FileUtil.loadmodel(trackScalerFile)
albumModel = FileUtil.loadmodel(albumModelFile)
albumScaler = FileUtil.loadmodel(albumScalerFile)


# 2. Hàm chuẩn hóa dữ liệu mới bằng đúng min, max lúc train rồi dự đoán cụm
def predictCluster(model, scaler, df):  3 usages
    columns = scaler["columns"]
    dfScaled = (df[columns] - scaler["min"]) / (scaler["max"] - scaler["min"])
    return model.predict(dfScaled.values)

```

Hình 7.1. Tải mô hình, chuẩn hóa theo min, max đã lưu và dự đoán cụm cho 59 khách

7.2. Xác định cụm và gợi ý album cho một khách

Ví dụ với khách 12: theo mô hình track, khách thuộc cụm 3 (tập trung ít thể loại); theo mô hình album, khách thuộc cụm 2 (theo album được nhiều người mua). Hai kết quả nhất quán với các giá trị đã tính ở mục 4.1: GenreCount = 5, AlbumCount = 18, AlbumPopularity = 5,83.

Gợi ý album dựa trên cụm album của khách và gồm bốn bước:

1. Lấy 25 khách thuộc cụm 2 (bao gồm cả khách 12).
2. Lấy tất cả album mà 25 khách này đã mua, bỏ đi 18 album khách 12 đã có.
3. Với mỗi album còn lại, đếm số khách trong cụm đã mua nó (cột CustomersInCluster).
4. Sắp xếp giảm dần theo số khách. Album có cùng số khách được xếp theo tên, để kết quả không đổi giữa các lần chạy. Lấy 5 album đầu.

Kết quả đứng đầu là *Greatest Hits* của Lenny Kravitz, được 10/25 khách cùng cụm mua. Bốn album tiếp theo mỗi album được 5 khách mua. Cách gợi ý này không cần hai khách cùng mua một track cụ thể, nên tránh được vấn đề dữ liệu thưa đã nêu ở mục 1.3.

Bốn bước trên nằm trong hàm recommendAlbums của ClusterUtil.py (Hình 7.2), nên console, trang HTML và giao diện đều cho cùng một danh sách gợi ý. Khi hai album có cùng số khách mua, hàm sắp tiếp theo tên album; nhờ vậy thứ tự gợi ý không thay đổi giữa các lần chạy.

```python
# 8. Thống kê trung bình các thuộc tính theo từng cụm (để đặt tên / mô tả cụm)
#    Cụm được đánh số từ 1 cho khớp với chú thích 'Cluster 1, 2, ...' trên biểu đồ
def showClusterSummary(df, columns):  3 usages
    dfShow = df.copy()
    dfShow["cluster"] = dfShow["cluster"] + 1
    summary = dfShow.groupby("cluster")[columns].mean().round(2)
    summary["NumberOfCustomers"] = dfShow.groupby("cluster")["CustomerId"].count()
    print(summary)
    return summary

# 9. Bài tập 1 (slide 07): in danh sách chi tiết Customer của từng cụm ra màn hình console
def showCustomerDetailByCluster(conn, df, cluster):  3 usages
    sql = "select CustomerId, FirstName, LastName, Company, City, Country, Email from Customer"
    dfCustomer = queryDataset(conn, sql)
    dfDetail = dfCustomer.merge(df[["CustomerId", "cluster"]], on="CustomerId")
    dfDetail["cluster"] = dfDetail["cluster"] + 1
    for i in range(cluster):
        dfCluster = dfDetail[dfDetail["cluster"] == i + 1]
        print("-" * 100)
        print(f"Cluster {i + 1}: có {len(dfCluster)} customer")
        print(dfCluster.to_string(index=False))

# 10. Gợi ý album: album được nhiều customer cùng cụm mua nhất mà customer này chưa mua
def recommendAlbums(dfAlbum, sameClusterIds, customerId, topN=5):  2 usages
    boughtAlbumIds = dfAlbum[dfAlbum["CustomerId"] == customerId]["AlbumId"].unique()
    dfCandidate = dfAlbum[dfAlbum["CustomerId"].isin(sameClusterIds) &
                          ~dfAlbum["AlbumId"].isin(boughtAlbumIds)]
    dfRecommend = dfCandidate.groupby(["AlbumId", "Title", "Artist"])["CustomerId"].nunique() \
        .reset_index(name="CustomersInCluster") \
        .sort_values(["CustomersInCluster", "Title"], ascending=[False, True])
    return dfRecommend.head(topN)

# 11. Bảng chi tiết để xuất Excel / HTML: thông tin customer + thuộc tính + cụm (đánh số từ 1) + tên cụm
#     df phải có cột "cluster" (nhãn gốc của KMeans, bắt đầu từ 0)
def getClusterDetail(conn, df, columns):  4 usages
    names = nameClusters(df, columns)
    dfDetail = getCustomerInfo(conn).merge(df, on="CustomerId")
    dfDetail["ClusterName"] = dfDetail["cluster"].map(lambda c: names[c])
    dfDetail["cluster"] = dfDetail["cluster"] + 1
    return dfDetail.sort_values(["cluster", "CustomerId"]).reset_index(drop=True)
```

Hình 7.2. Các hàm showClusterSummary, showCustomerDetailByCluster, recommendAlbums và getClusterDetail

```python
# 4. Gợi ý album: album được nhiều customer cùng cụm ALBUM mua nhất mà customer này chưa mua
dfAlbum = getAlbumPurchaseDataset(conn)
sameClusterIds = df[df["AlbumCluster"] == albumCluster]["CustomerId"]
dfRecommend = recommendAlbums(dfAlbum, sameClusterIds, customerId, topN: 5)
print("Top 5 album gợi ý cho customer", customerId)
print(dfRecommend.to_string(index=False))

# 5. Dự đoán cụm cho 1 customer MỚI (tự nhập các thuộc tính)
genreCount = float(input("Nhập GenreCount của customer mới (vd 5): "))
avgMinutes = float(input("Nhập AvgMinutes của customer mới (vd 4.5): "))
dfNew = pd.DataFrame( data: [[genreCount, avgMinutes]], columns=["GenreCount", "AvgMinutes"])
newCluster = predictCluster(trackModel, trackScaler, dfNew)
print("Customer mới thuộc cụm TRACK: Cluster", newCluster[0] + 1)

closeConnection(conn)
```

Hình 7.3. Gọi recommendAlbums cho khách đang xét và dự đoán cụm cho khách mới

```
Nhập CustomerId (1 - 59): 12
 CustomerId  GenreCount  AvgMinutes  AlbumCount  AlbumPopularity  TrackCluster  AlbumCluster
         12           5        3.82          18             5.83             3             2
Cụm theo quan tâm TRACK: Cluster 3
Cụm theo quan tâm ALBUM: Cluster 2
Top 5 album gợi ý cho customer 12
 AlbumId                                  Title                      Artist  CustomersInCluster
     141                          Greatest Hits               Lenny Kravitz                  10
      89                         American Idiot                   Green Day                   5
     253 Battlestar Galactica (Classic), Season 1 Battlestar Galactica (Classic)              5
     227           Battlestar Galactica, Season 3          Battlestar Galactica              5
     193                    Blood Sugar Sex Magik         Red Hot Chili Peppers              5
Nhập GenreCount của customer mới (vd 5): 5
Nhập AvgMinutes của customer mới (vd 4.5): 4.5
Customer mới thuộc cụm TRACK: Cluster 3
```

Hình 7.4. Kết quả chạy using_model.py với CustomerId = 12 và khách mới GenreCount = 5, AvgMinutes = 4,5.

7.3. Dự đoán cụm cho khách mới

Khách mới có GenreCount = 5 và AvgMinutes = 4,5 được chuẩn hóa thành (0,125; 0,0724). Khoảng cách Euclid tới ba tâm cụm của mô hình track (Hình 6.2) như sau:

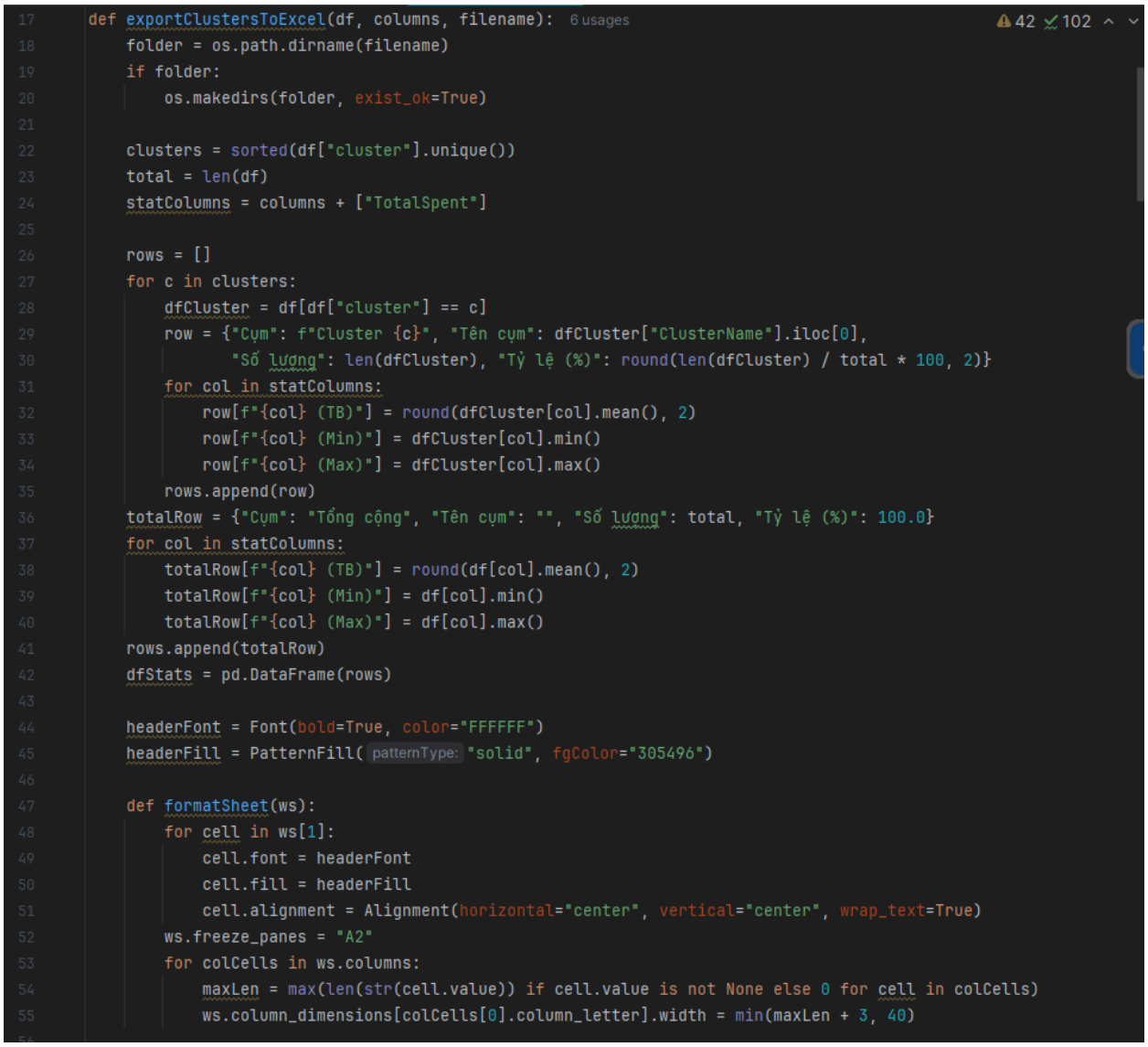
Cụm	Tâm cụm (đã chuẩn hóa)	Khoảng cách
1	(0,5972; 0,1620)	0,4807
2	(0,6125; 0,6245)	0,7365
3	(0,2782; 0,0996)	**0,1556**

Khoảng cách tới tâm cụm 3 nhỏ nhất, nên model.predict trả về cụm 3. Kết quả này khớp với dòng cuối ở Hình 7.4. Phép tính giống bước 2 của ví dụ tính tay trong bài giảng: khách được gán vào cụm có vector trọng tâm gần nhất.

7.4. Xuất kết quả ra Excel và trang HTML

Kết quả in trên console chỉ xem được lúc chạy chương trình. Để người không chạy code, chẳng hạn bộ phận marketing, cũng đọc được, bài làm xuất kết quả ra hai dạng: một file Excel cho mỗi thực nghiệm và một trang HTML tổng hợp cả ba mô hình. Cả hai dùng chung bảng chi tiết do getClusterDetail tạo ra, gồm thông tin khách, các thuộc tính, số cụm đánh từ 1 và tên cụm tự động (mục 6.4).

File Excel. Hàm exportClustersToExcel (Hình 7.5) được gọi ở cuối mỗi file CustomerCluster*.py. Mỗi cụm được ghi vào một sheet riêng tên "Cluster 1", "Cluster 2", … chứa danh sách khách của cụm. Sheet "Thống kê" ghi cho từng cụm số khách, tỷ lệ, giá trị trung bình, nhỏ nhất và lớn nhất của từng thuộc tính và của tổng chi tiêu; dòng cuối là dòng "Tổng cộng" tính trên cả 59 khách và được in đậm. Hàng tiêu đề được tô nền, cố định khi cuộn, và độ rộng cột tự co theo nội dung. Ba file được ghi vào results/excel/: CustomerCluster1_Track.xlsx, CustomerCluster2_Album.xlsx và CustomerCluster3_TrackAlbum.xlsx (Hình 7.6).



```python
def exportClustersToExcel(df, columns, filename):  6 usages
    folder = os.path.dirname(filename)
    if folder:
        os.makedirs(folder, exist_ok=True)

    clusters = sorted(df["cluster"].unique())
    total = len(df)
    statColumns = columns + ["TotalSpent"]

    rows = []
    for c in clusters:
        dfCluster = df[df["cluster"] == c]
        row = {"Cụm": f"Cluster {c}", "Tên cụm": dfCluster["ClusterName"].iloc[0],
               "Số lượng": len(dfCluster), "Tỷ lệ (%)": round(len(dfCluster) / total * 100, 2)}
        for col in statColumns:
            row[f"{col} (TB)"] = round(dfCluster[col].mean(), 2)
            row[f"{col} (Min)"] = dfCluster[col].min()
            row[f"{col} (Max)"] = dfCluster[col].max()
        rows.append(row)
    totalRow = {"Cụm": "Tổng cộng", "Tên cụm": "", "Số lượng": total, "Tỷ lệ (%)": 100.0}
    for col in statColumns:
        totalRow[f"{col} (TB)"] = round(df[col].mean(), 2)
        totalRow[f"{col} (Min)"] = df[col].min()
        totalRow[f"{col} (Max)"] = df[col].max()
    rows.append(totalRow)
    dfStats = pd.DataFrame(rows)

    headerFont = Font(bold=True, color="FFFFFF")
    headerFill = PatternFill( patternType: "solid", fgColor="305496")

    def formatSheet(ws):
        for cell in ws[1]:
            cell.font = headerFont
            cell.fill = headerFill
            cell.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)
        ws.freeze_panes = "A2"
        for colCells in ws.columns:
            maxLen = max(len(str(cell.value)) if cell.value is not None else 0 for cell in colCells)
            ws.column_dimensions[colCells[0].column_letter].width = min(maxLen + 3, 40)
```

Hình 7.5. Hàm exportClustersToExcel: mỗi cụm một sheet và sheet Thống kê có dòng tổng

```
Đã xuất Excel: Chinook_Customer_Clustering/results/excel/CustomerCluster1_Track.xlsx
Đã lưu mô hình: Chinook_Customer_Clustering/Trained/kmeans_track_model.zip
```

Hình 7.6. Hai dòng cuối khi chạy CustomerCluster1.py: file Excel được ghi vào results/excel/, mô hình được lưu vào Trained/.

Trang HTML. File export_results.py (Hình 7.7) không huấn luyện lại mà tải ba mô hình trong Trained/, chuẩn hóa dữ liệu bằng min, max đã lưu rồi dự đoán cụm. Với mỗi mô hình, chương trình tính lại đường Elbow bằng findElbow, lấy bảng chi tiết bằng getClusterDetail, rồi thống kê từng cụm: số khách, trung bình thuộc tính, chi tiêu trung

bình, 5 thể loại và 5 album được mua nhiều nhất trong cụm. Phần cuối tạo bảng tra cứu 59 khách, ghi cụm của khách ở cả ba mô hình và 5 album gợi ý theo mô hình album. Tên khách, tên album và tên nghệ sĩ được escape bằng html.escape trước khi đưa vào trang, vì bảng tra cứu cho phép thẻ HTML để xuống dòng giữa các album. Dòng in ra khi chạy (Hình 7.8) cho thấy K của mô hình đã lưu trùng với K do Kneedle chọn, và Silhouette trùng với Chương 6.

```
for exp in EXPERIMENTS:
    modelFile, scalerFile = getModelPaths(exp["file"])
    if not os.path.exists(modelFile) or not os.path.exists(scalerFile):
        raise SystemExit("Chưa có " + modelFile + " -> hãy chạy CustomerCluster1.py, 2, 3 trước")
    model = FileUtil.loadmodel(modelFile)
    scaler = FileUtil.loadmodel(scalerFile)
    columns = scaler["columns"]
    # 1. Chuẩn hóa bằng min, max đã lưu rồi dự đoán cụm cho 59 customer
    X = ((dfFeature[columns] - scaler["min"]) / (scaler["max"] - scaler["min"])).values
    df2 = dfFeature.copy()
    df2["cluster"] = model.predict(X)
    # 2. Elbow + Kneedle (để vẽ lại đồ thị chọn K)
    kRange, inertia, elbowZone, optimalK = findElbow(X)
    # 3. Bảng chi tiết customer + tên cụm
    dfDetail = getClusterDetail(conn, df2, columns)
    names = dict(zip(dfDetail["cluster"], dfDetail["ClusterName"]))
    # 4. Thống kê từng cụm, 5 thể loại và 5 album được mua nhiều nhất trong cụm
    dfAlbumCluster = dfAlbum.merge(dfDetail[["CustomerId", "cluster"]], on="CustomerId")
    rows, topGenres, topAlbums = [], {}, {}
    for c in sorted(dfDetail["cluster"].unique()):
        dfC = dfDetail[dfDetail["cluster"] == c]
        dfA = dfAlbumCluster[dfAlbumCluster["cluster"] == c]
        genreShare = (dfA["Genre"].value_counts(normalize=True) * 100).round(1)
        topGenres[c] = genreShare.head(5).rename_axis("Genre").reset_index(name="Tỷ lệ lượt mua (%)")
        topAlbums[c] = dfA.groupby(["Title", "Artist"])["CustomerId"].nunique() \
            .reset_index(name="Số khách") \
            .sort_values(["Số khách", "Title"], ascending=[False, True]).head(5)
        row = {"Cụm": f"Cluster {c}", "Tên cụm": names[c], "Số khách": len(dfC),
               "Tỷ lệ (%)": round(len(dfC) / len(dfDetail) * 100, 1)}
        for col in columns:
            row[f"{col} TB"] = round(dfC[col].mean(), 2)
        row["Chi tiêu TB (USD)"] = round(dfC["TotalSpent"].mean(), 2)
        row["Thể loại chính"] = ", ".join(f"{g} {p:.0f}%" for g, p in genreShare.head(3).items())
        rows.append(row)
    experiments.append({
        "name": exp["name"], "columns": columns, "df": dfDetail, "names": names,
        "k": model.n_clusters, "kRange": kRange.tolist(), "inertia": inertia.tolist(),
        "elbowZone": elbowZone.tolist(), "wcss": model.inertia_,
        "silhouette": silhouette_score(X, df2["cluster"]),
        "summary": pd.DataFrame(rows), "topGenres": topGenres, "topAlbums": topAlbums})
    print(exp["name"], "| K =", model.n_clusters, "| Kneedle K =", optimalK,
          "| Silhouette =", round(experiments[-1]["silhouette"], 4))
```

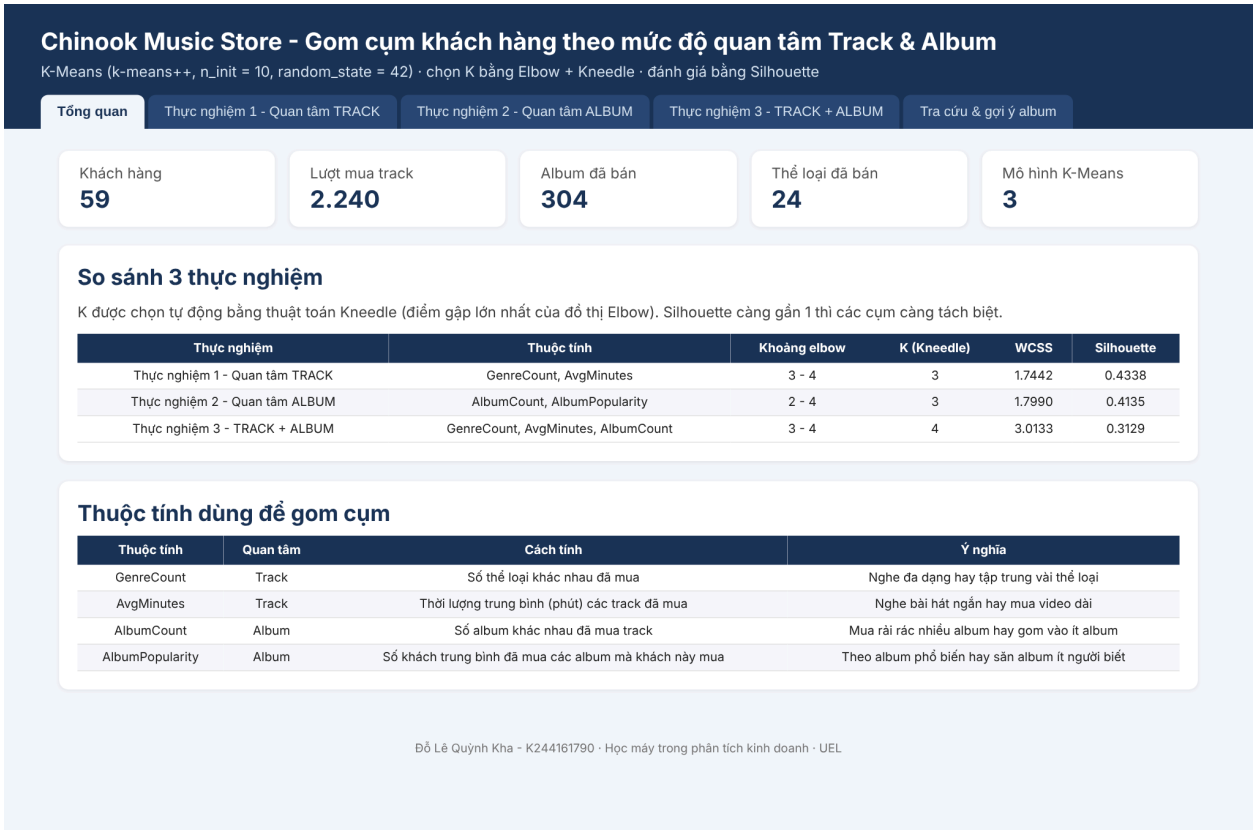
Hình 7.7. Tải ba mô hình và chuẩn bị dữ liệu cho trang HTML (Clustering/export_results.py, dòng 19–66).

```
Thực nghiệm 1 - Quan tâm TRACK | K = 3 | Kneedle K = 3 | Silhouette = 0.4338
Thực nghiệm 2 - Quan tâm ALBUM | K = 3 | Kneedle K = 3 | Silhouette = 0.4135
Thực nghiệm 3 - TRACK + ALBUM | K = 4 | Kneedle K = 4 | Silhouette = 0.3129
Đã xuất HTML: Chinook_Customer_Clustering/results/html/index.html
```

Hình 7.8. Kết quả chạy export_results.py.

Hàm exportClustersToHtml ghép các bảng và biểu đồ plotly thành một file results/html/index.html duy nhất. Thư viện plotly.js được nhúng một lần trong phần <head>, các biểu đồ không nhúng lại, nên trang mở được cả khi không có mạng và file nặng khoảng 5 MB. Trang có năm tab:

- **Tổng quan** (Hình 7.9): số khách, lượt mua, số album và thể loại đã bán; bảng so sánh ba thực nghiệm (khoảng elbow, K, WCSS, Silhouette) và bảng giải thích bốn thuộc tính.
- **Ba tab thực nghiệm**: đồ thị Elbow có tô khoảng elbow đặt cạnh biểu đồ tròn tỷ lệ khách theo cụm (Hình 7.10); biểu đồ cụm 2D, hoặc 3D với thực nghiệm 3 (Hình 7.11, 7.14); bảng thống kê cụm; và một khung cho mỗi cụm gồm 5 thể loại, 5 album được mua nhiều nhất và danh sách khách có thể mở ra khi cần (Hình 7.12, 7.13).
- **Tra cứu & gợi ý album** (Hình 7.15): bảng 59 khách có ô lọc theo tên, quốc gia hoặc CustomerId.



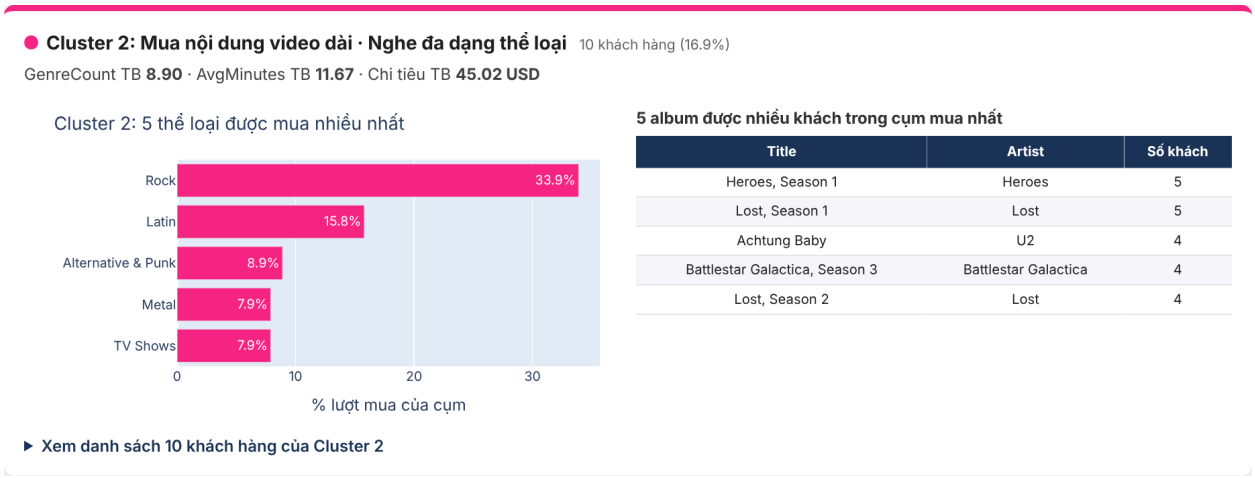
Hình 7.9. Tab Tổng quan của results/html/index.html.



Hình 7.10. Thực nghiệm 1 trên trang HTML: đồ thị Elbow + Kneedle (K = 3) và tỷ lệ khách của ba cụm 18 / 10 / 31.

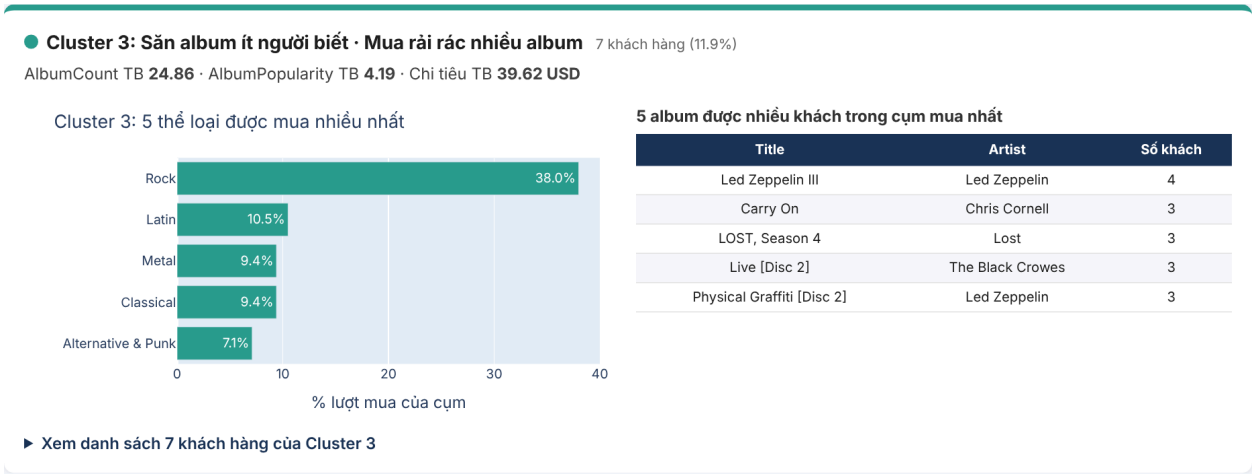


Hình 7.11. Biểu đồ cụm của thực nghiệm 1 trên trang HTML.

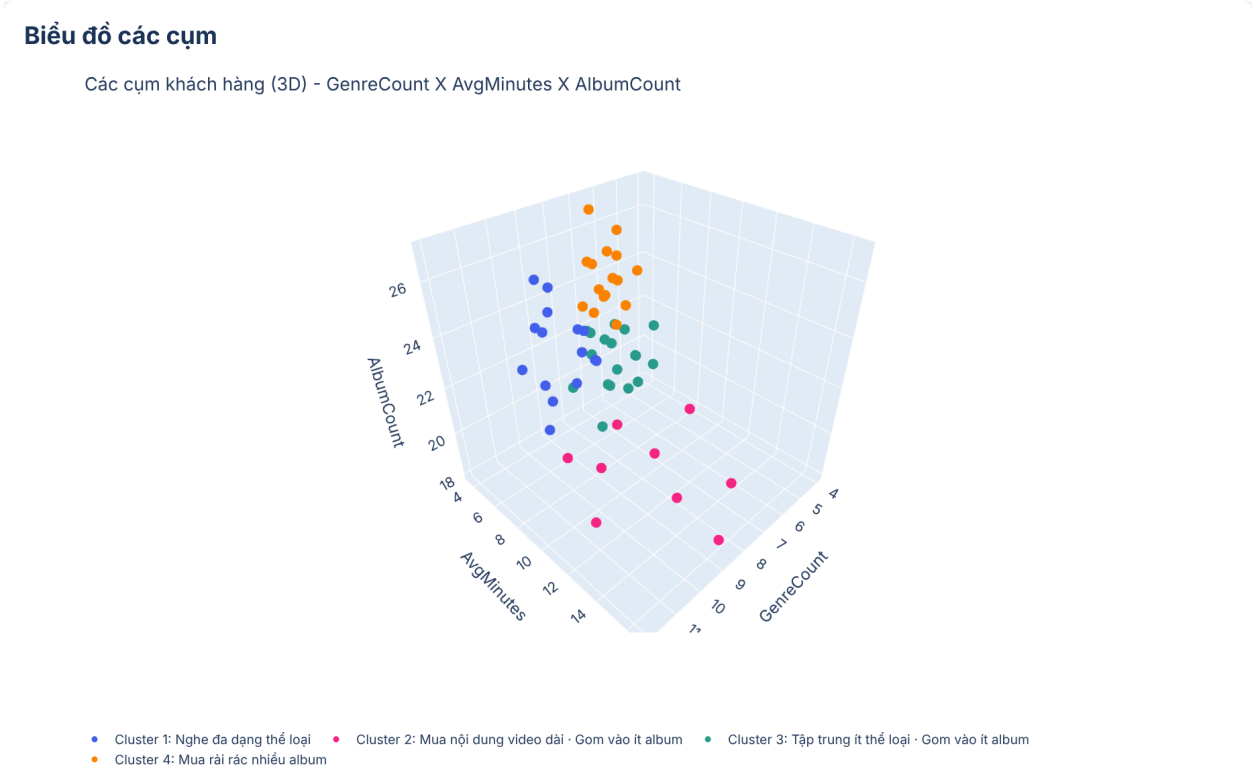


Title	Artist	Số khách
Heroes, Season 1	Heroes	5
Lost, Season 1	Lost	5
Achtung Baby	U2	4
Battlestar Galactica, Season 3	Battlestar Galactica	4
Lost, Season 2	Lost	4

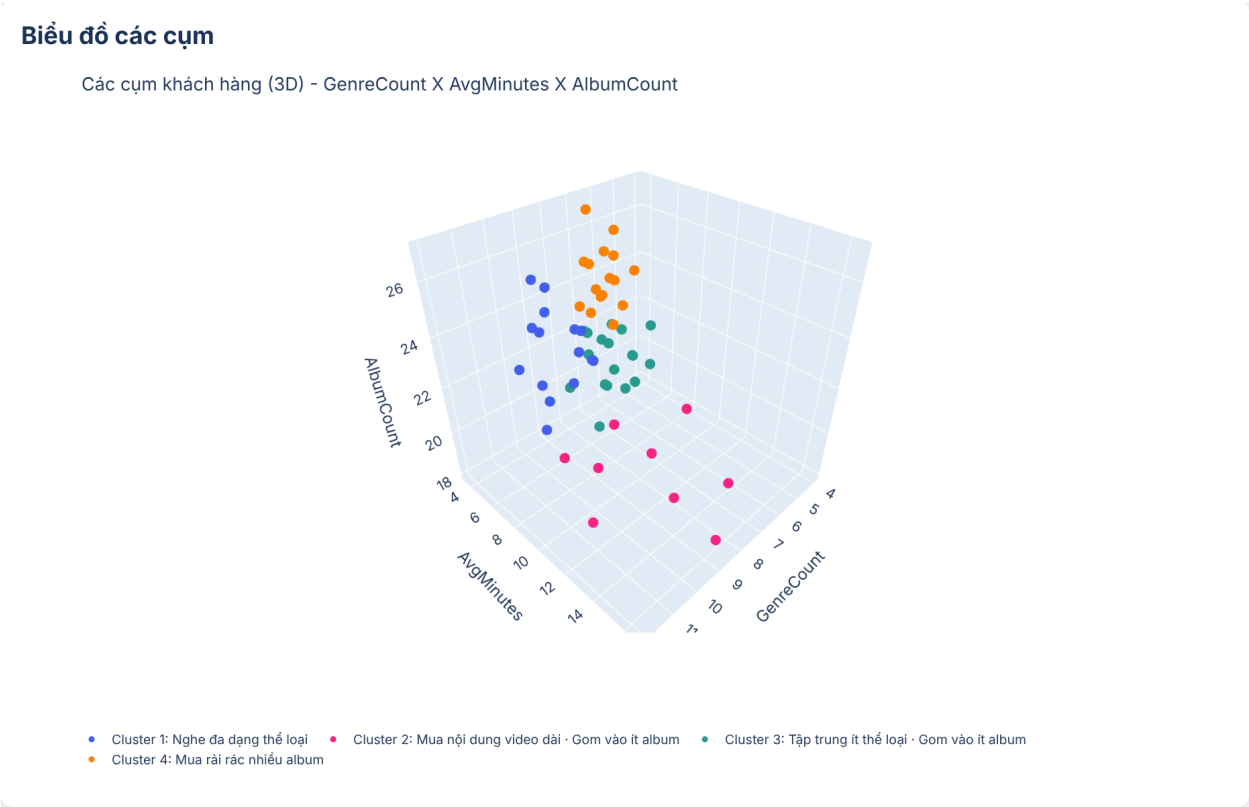
Hình 7.12. Khung chi tiết cụm 2 của thực nghiệm 1. TV Shows xuất hiện trong 5 thể loại đứng đầu, và bốn trong năm album được mua nhiều nhất là phim bộ (Heroes, Lost, Battlestar Galactica), khớp với tên "Mua nội dung video dài".



Hình 7.13. Khung chi tiết cụm 3 của thực nghiệm 2 (7 khách, AlbumPopularity trung bình 4,19). Album được mua nhiều nhất chỉ có 4/7 khách mua, và Classical chiếm 9,4% lượt mua.



Hình 7.14. Biểu đồ 3D của thực nghiệm 3 trên trang HTML; có thể xoay và di chuột để xem từng khách.



Hình 7.15. Tab Tra cứu & gợi ý album: cụm của từng khách ở ba mô hình và 5 album gợi ý.

Bảng tra cứu cho thấy gợi ý thay đổi theo cụm album của khách. Khách 3 (François Tremblay) thuộc cụm 3 của thực nghiệm 2, nên được gợi ý Led Zeppelin III, Carry On và LOST, Season 4, đúng là các album đứng đầu của cụm này ở Hình 7.13. Khách 1 và khách 5 cùng thuộc cụm 1 nên danh sách của hai người trùng nhau ở các album cả hai đều chưa mua, như Unplugged và The Cream Of Clapton của Eric Clapton.

CHƯƠNG 8. GIAO DIỆN PYQT6 TÍCH HỢP MÔ HÌNH

Giao diện gom toàn bộ quy trình ở Chương 5–7 vào một cửa sổ. Người dùng có thể nạp dữ liệu, vẽ Elbow, huấn luyện, lưu và tải mô hình, tra cứu khách và dự đoán khách mới mà không phải chạy từng file. Cách tổ chức giống SalesDatabaseProject trên lớp. ChinookMainWindow.ui được thiết kế bằng Qt Designer, ChinookMainWindow.py do PyUIC sinh ra. Lớp ChinookMainWindowEx kế thừa Ui_MainWindow và viết xử lý sự kiện. main.py khởi động ứng dụng giống testSalesMainWindowEx.py.

8.1. Bố cục màn hình

Vùng (QGroupBox)	Widget chính	Chức năng
1. Dữ liệu và mô hình	lineEditDatabase, comboBoxModel, spinBoxK, năm nút Load Data / Elbow / Train / Save / Load, labelStatus	Chọn dữ liệu, chọn mô hình (3 lựa chọn ứng với 3 thực nghiệm), chọn K, hiển thị Silhouette
2. Danh sách Customer và cụm	tableWidgetCustomer	59 khách với bốn thuộc tính và cột Cluster
3. Trực quan hóa cụm	verticalLayoutPlot (FigureCanvas + NavigationToolbar)	Đồ thị Elbow, biểu đồ cụm 2D hoặc 3D
4. Tra cứu Customer và gợi ý Album	spinBoxCustomerId, pushButtonSearch, labelSearchResult, tableWidgetRecommend	Cụm của khách và 5 album gợi ý
5. Dự đoán cụm cho Customer mới	lineEditGenreCount, lineEditAvgMinutes, pushButtonPredict, lineEditPredictResult	Dự đoán cụm theo mô hình track
6. Thống kê theo cụm	tableWidgetSummary	Trung bình thuộc tính và số khách mỗi cụm

8.2. Xử lý sự kiện

Mỗi nút được nối với một hàm trong setupSignalAndSlot. Các hàm này gọi lại đúng những hàm đã dùng ở phần console (getCustomerFeatures, minMaxScale, runKMeans, FileUtil). Vì vậy kết quả trên giao diện trùng khớp với Chương 6: Silhouette 0,434 cho mô hình track, 0,414 cho mô hình album, 0,313 cho mô hình ba thuộc tính. Giao diện và

các file CustomerCluster*.py cũng dùng chung tên file trong Trained/. Mô hình huấn luyện ở console có thể mở bằng nút Load Model và ngược lại.

```
43      def setupSignalAndSlot(self):  1 usage
44          self.pushButtonLoadData.clicked.connect(self.loadData)
45          self.pushButtonElbow.clicked.connect(self.showElbow)
46          self.pushButtonTrain.clicked.connect(self.trainModel)
47          self.pushButtonSaveModel.clicked.connect(self.saveModel)
48          self.pushButtonLoadModel.clicked.connect(self.loadModel)
49          self.pushButtonSearch.clicked.connect(self.searchCustomer)
50          self.pushButtonPredict.clicked.connect(self.predictNewCustomer)
51          self.comboBoxModel.currentIndexChanged.connect(self.changeModel)
52
53      def setupPlot(self):  1 usage
54          self.figure = plt.figure()
55          self.canvas = FigureCanvas(self.figure)
56          self.toolbar = NavigationToolbar(self.canvas, self.MainWindow)
57
58          self.verticalLayoutPlot.addWidget(self.toolbar)
59          self.verticalLayoutPlot.addWidget(self.canvas)
```

Hình 8.1. Nối tín hiệu với hàm xử lý và nhúng biểu đồ matplotlib vào verticalLayoutPlot

Nút / sự kiện	**Hàm xử lý**	**Việc thực hiện**
Load Data	loadData	Kiểm tra file tồn tại, tính thuộc tính, nạp bảng khách và bảng album đã mua
Elbow	showElbow	Chạy K-Means với K = 1–10, vẽ WCSS vào vùng 3
Train Model	trainModel	Chuẩn hóa, huấn luyện với K từ spinBoxK, cập nhật bảng, biểu đồ, thống kê, Silhouette
Save Model / Load Model	saveModel / loadModel	Ghi hoặc đọc hai file .zip của mô hình đang chọn
Xem cụm & Gợi ý	searchCustomer	Hiện cụm, tô vàng dòng của khách, khoanh tròn khách trên biểu đồ, liệt kê 5 album
Predict Cluster	predictNewCustomer	Chuẩn hóa hai giá trị nhập vào, dự đoán cụm bằng mô hình track
Đổi comboBoxModel	changeModel	Xóa mô hình đang dùng, yêu cầu huấn luyện hoặc tải lại

Các thao tác sai thứ tự được chặn bằng QMessageBox thay vì để chương trình dừng lỗi. Ví dụ: bấm Train trước khi nạp dữ liệu, bấm tra cứu khi chưa có mô hình, nhập chữ vào ô GenreCount, hoặc tải mô hình chưa từng được lưu.

```
163     def trainModel(self):  1 usage
164         if not self.checkData():
165             return
166         columns = self.getColumns()
167         cluster = self.spinBoxK.value()
168         dfScaled, minValues, maxValues = minMaxScale(self.df, columns)
169         X = dfScaled.loc[:, columns].values
170 
171         y_kmeans, centroids, labels, model = runKMeans(X, cluster)
172         self.model = model
173         self.scaler = {"columns": columns, "min": minValues, "max": maxValues}
174         self.showResult()
175         self.labelStatus.setText(f"Train xong: K = {cluster}  |  Silhouette = {self.silhouette:.3f}")
176 
177     # Chuẩn hóa bằng min, max đã lưu rồi predict cụm cho 59 customer
178     def predictAll(self):  1 usage
179         columns = self.scaler["columns"]
180         dfScaled = (self.df[columns] - self.scaler["min"]) / (self.scaler["max"] - self.scaler["min"])
181         X = dfScaled.values
182         labels = self.model.predict(X)
183         self.silhouette = silhouette_score(X, labels)
184         self.df["Cluster"] = labels + 1          # hiển thị cụm bắt đầu từ 1 giống thầy
185 
186     def showResult(self, highlightCustomerId=None):  2 usages
187         self.predictAll()
188         self.showDataIntoTableWidget(self.tableWidgetCustomer, self.df)
189         self.showSummary()
190         self.visualizeKMeans(highlightCustomerId)
```

Hình 8.2. Hàm trainModel và predictAll: huấn luyện, lưu min/max, dự đoán cụm và tính Silhouette



```
    # ------------------------------------------------------------
    # 5. Tra cứu Customer: xem cụm + gợi ý album
    #    Gợi ý = album được nhiều customer CÙNG CỤM mua nhất mà customer này chưa mua
    # ------------------------------------------------------------
    def searchCustomer(self):  1 usage
        if not self.checkData() or not self.checkModel():
            return
        customerId = self.spinBoxCustomerId.value()
        dfOne = self.df[self.df["CustomerId"] == customerId]
        cluster = dfOne["Cluster"].values[0]
        self.labelSearchResult.setText(f"{dfOne['FullName'].values[0]} ({dfOne['Country'].values[0]})"
                                       f"  →  Cluster {cluster}")

        sameClusterIds = self.df[self.df["Cluster"] == cluster]["CustomerId"]
        boughtAlbumIds = self.dfAlbum[self.dfAlbum["CustomerId"] == customerId]["AlbumId"].unique()
        dfCandidate = self.dfAlbum[self.dfAlbum["CustomerId"].isin(sameClusterIds) &
                                   ~self.dfAlbum["AlbumId"].isin(boughtAlbumIds)]
        dfRecommend = dfCandidate.groupby(["Title", "Artist"])["CustomerId"].nunique() \
            .reset_index(name="CustomersInCluster") \
            .sort_values(["CustomersInCluster", "Title"], ascending=[False, True]).head(5)
        self.showDataIntoTableWidget(self.tableWidgetRecommend, dfRecommend)

        # tô vàng dòng customer trong bảng và khoanh tròn trên biểu đồ
        self.showDataIntoTableWidget(self.tableWidgetCustomer, self.df)
        row = dfOne.index[0]
        for j in range(self.tableWidgetCustomer.columnCount()):
            self.tableWidgetCustomer.item(row, j).setBackground(QColor("yellow"))
        self.tableWidgetCustomer.scrollToItem(self.tableWidgetCustomer.item(row, 0))
        self.visualizeKMeans(customerId)
```

Hình 8.3. Hàm searchCustomer: gọi recommendAlbums, đánh dấu khách trên bảng và trên biểu đồ

8.3. Kết quả chạy giao diện

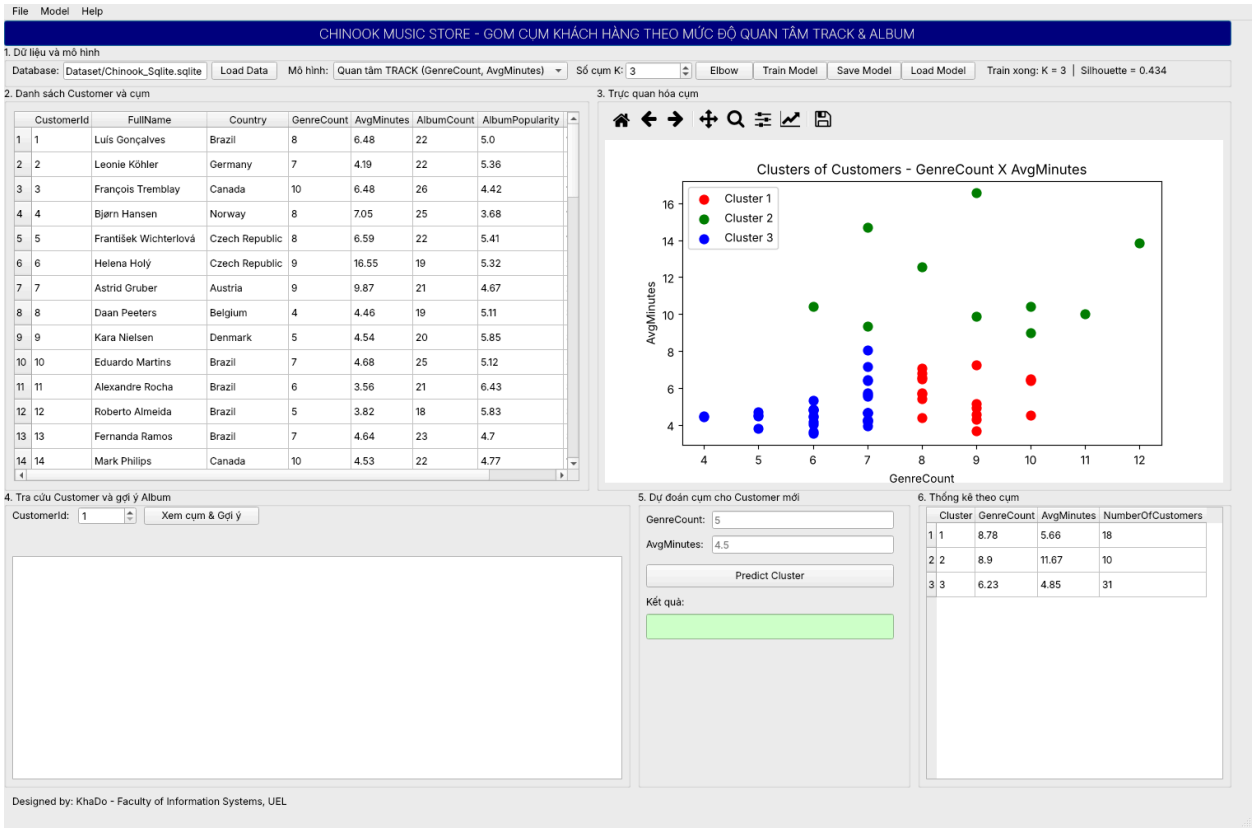
```
    # ------------------------------------------------------------
    # 5. Tra cứu Customer: xem cụm + gợi ý album
    #    Gợi ý = album được nhiều customer CÙNG CỤM mua nhất mà customer này chưa mua
    # ------------------------------------------------------------
    def searchCustomer(self):  1 usage
        if not self.checkData() or not self.checkModel():
            return
        customerId = self.spinBoxCustomerId.value()
        dfOne = self.df[self.df["CustomerId"] == customerId]
        cluster = dfOne["Cluster"].values[0]
        self.labelSearchResult.setText(f"{dfOne['FullName'].values[0]} ({dfOne['Country'].values[0]})"
                                       f"  →  Cluster {cluster}")

        sameClusterIds = self.df[self.df["Cluster"] == cluster]["CustomerId"]
        boughtAlbumIds = self.dfAlbum[self.dfAlbum["CustomerId"] == customerId]["AlbumId"].unique()
        dfCandidate = self.dfAlbum[self.dfAlbum["CustomerId"].isin(sameClusterIds) &
                                   ~self.dfAlbum["AlbumId"].isin(boughtAlbumIds)]
        dfRecommend = dfCandidate.groupby(["Title", "Artist"])["CustomerId"].nunique() \
            .reset_index(name="CustomersInCluster") \
            .sort_values(["CustomersInCluster", "Title"], ascending=[False, True]).head(5)
        self.showDataIntoTableWidget(self.tableWidgetRecommend, dfRecommend)

        # tô vàng dòng customer trong bảng và khoanh tròn trên biểu đồ
        self.showDataIntoTableWidget(self.tableWidgetCustomer, self.df)
        row = dfOne.index[0]
        for j in range(self.tableWidgetCustomer.columnCount()):
            self.tableWidgetCustomer.item(row, j).setBackground(QColor("yellow"))
        self.tableWidgetCustomer.scrollToItem(self.tableWidgetCustomer.item(row, 0))
        self.visualizeKMeans(customerId)

```

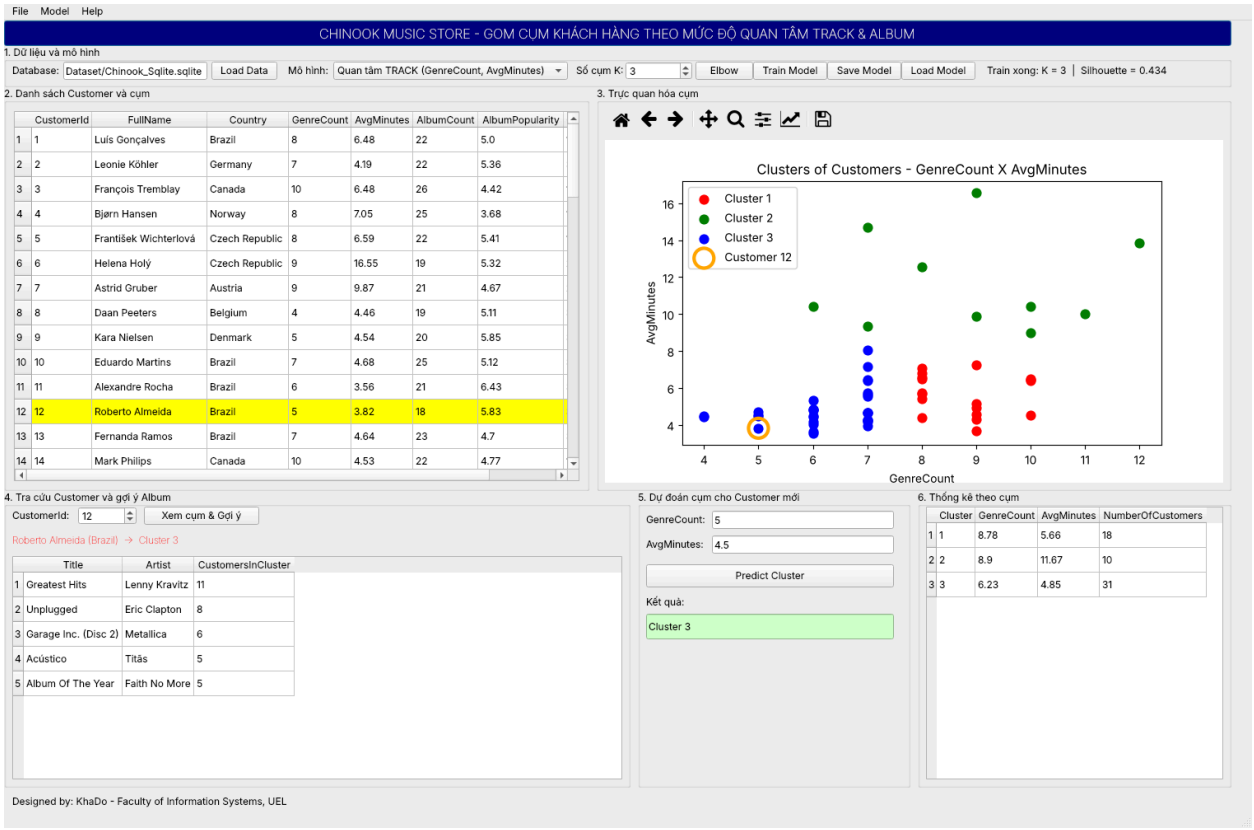
Hình 8.4. Sau khi bấm Load Data và Elbow với mô hình track: bảng 59 khách và đồ thị Elbow.



	CustomerId	FullName	Country	GenreCount	AvgMinutes	AlbumCount	AlbumPopularity
1	1	Luís Gonçalves	Brazil	8	6.48	22	5.0
2	2	Leonie Köhler	Germany	7	4.19	22	5.36
3	3	François Tremblay	Canada	10	6.48	26	4.42
4	4	Bjørn Hansen	Norway	8	7.05	25	3.68
5	5	František Wichterlová	Czech Republic	8	6.59	22	5.41
6	6	Helena Holý	Czech Republic	9	16.55	19	5.32
7	7	Astrid Gruber	Austria	9	9.87	21	4.67
8	8	Daan Peeters	Belgium	4	4.46	19	5.11
9	9	Kara Nielsen	Denmark	5	4.54	20	5.85
10	10	Eduardo Martins	Brazil	7	4.68	25	5.12
11	11	Alexandre Rocha	Brazil	6	3.56	21	6.43
12	12	Roberto Almeida	Brazil	5	3.82	18	5.83
13	13	Fernanda Ramos	Brazil	7	4.64	23	4.7
14	14	Mark Philips	Canada	10	4.53	22	4.77

	Cluster	GenreCount	AvgMinutes	NumberOfCustomers
1	1	8.78	5.66	18
2	2	8.9	11.67	10
3	3	6.23	4.85	31

Hình 8.5. Sau khi bấm Train Model với K = 3: cột Cluster, biểu đồ cụm, bảng thống kê và Silhouette = 0,434 trên thanh trạng thái.

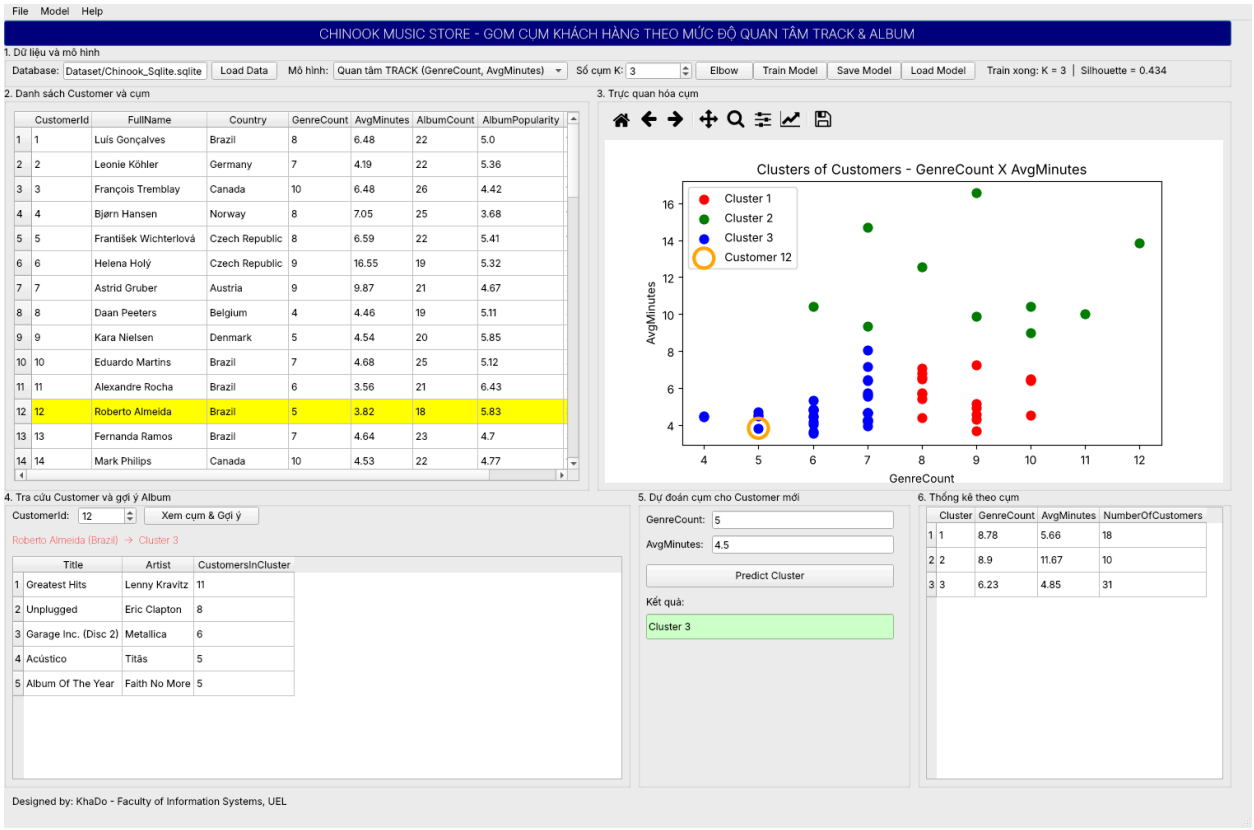


	CustomerId	FullName	Country	GenreCount	AvgMinutes	AlbumCount	AlbumPopularity
1	1	Luís Gonçalves	Brazil	8	6.48	22	5.0
2	2	Leonie Köhler	Germany	7	4.19	22	5.36
3	3	François Tremblay	Canada	10	6.48	26	4.42
4	4	Bjørn Hansen	Norway	8	7.05	25	3.68
5	5	František Wichterlová	Czech Republic	8	6.59	22	5.41
6	6	Helena Holý	Czech Republic	9	16.55	19	5.32
7	7	Astrid Gruber	Austria	9	9.87	21	4.67
8	8	Daan Peeters	Belgium	4	4.46	19	5.11
9	9	Kara Nielsen	Denmark	5	4.54	20	5.85
10	10	Eduardo Martins	Brazil	7	4.68	25	5.12
11	11	Alexandre Rocha	Brazil	6	3.56	21	6.43
12	12	Roberto Almeida	Brazil	5	3.82	18	5.83
13	13	Fernanda Ramos	Brazil	7	4.64	23	4.7
14	14	Mark Philips	Canada	10	4.53	22	4.77

	Title	Artist	CustomersInCluster
1	Greatest Hits	Lenny Kravitz	11
2	Unplugged	Eric Clapton	8
3	Garage Inc. (Disc 2)	Metallica	6
4	Acústico	Titãs	5
5	Album Of The Year	Faith No More	5

	Cluster	GenreCount	AvgMinutes	NumberOfCustomers
1	1	8.78	5.66	18
2	2	8.9	11.67	10
3	3	6.23	4.85	31

Hình 8.6. Tra cứu khách 12 (dòng tô vàng, điểm khoanh tròn cam), 5 album gợi ý và kết quả dự đoán khách mới "Cluster 3".

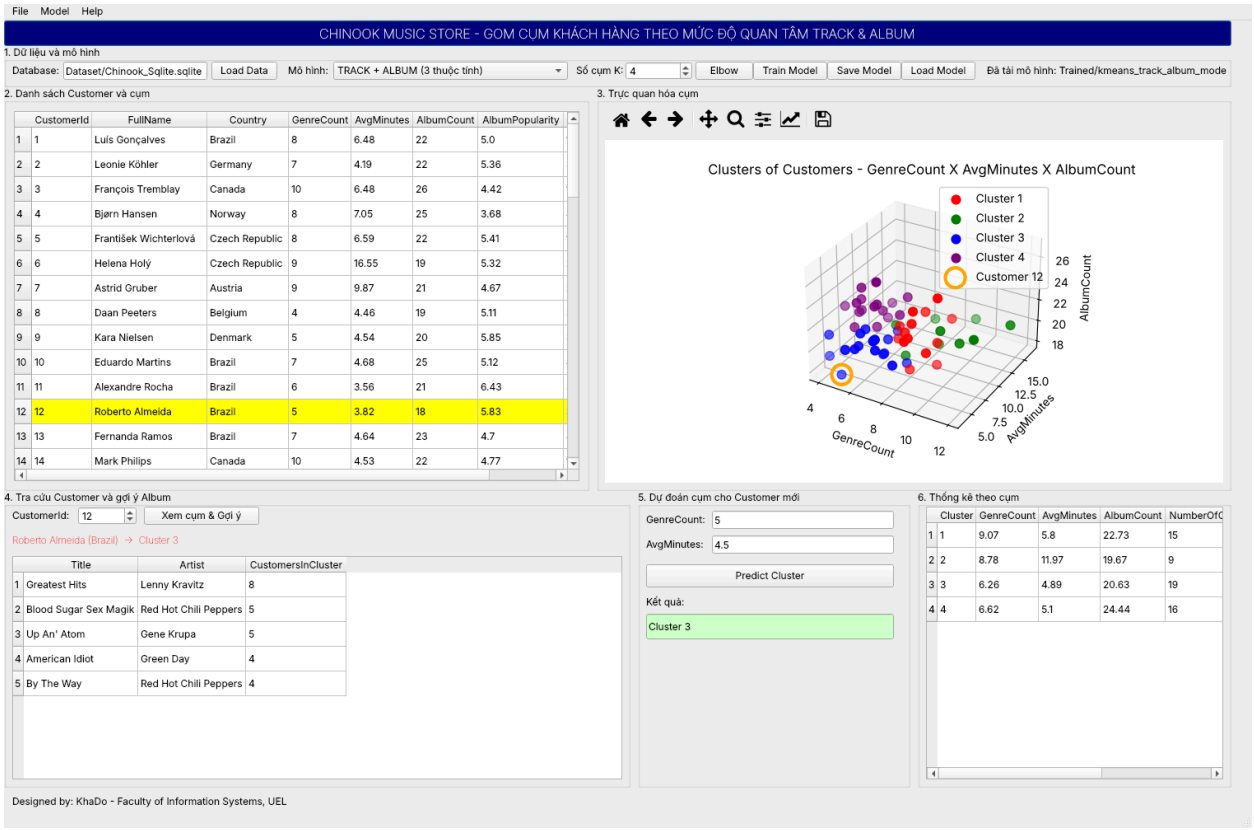


	CustomerId	FullName	Country	GenreCount	AvgMinutes	AlbumCount	AlbumPopularity
1	1	Luís Gonçalves	Brazil	8	6.48	22	5.0
2	2	Leonie Köhler	Germany	7	4.19	22	5.36
3	3	François Tremblay	Canada	10	6.48	26	4.42
4	4	Bjørn Hansen	Norway	8	7.05	25	3.68
5	5	František Wichterlová	Czech Republic	8	6.59	22	5.41
6	6	Helena Holý	Czech Republic	9	16.55	19	5.32
7	7	Astrid Gruber	Austria	9	9.87	21	4.67
8	8	Daan Peeters	Belgium	4	4.46	19	5.11
9	9	Kara Nielsen	Denmark	5	4.54	20	5.85
10	10	Eduardo Martins	Brazil	7	4.68	25	5.12
11	11	Alexandre Rocha	Brazil	6	3.56	21	6.43
12	12	Roberto Almeida	Brazil	5	3.82	18	5.83
13	13	Fernanda Ramos	Brazil	7	4.64	23	4.7
14	14	Mark Philips	Canada	10	4.53	22	4.77

	Title	Artist	CustomersInCluster
1	Greatest Hits	Lenny Kravitz	11
2	Unplugged	Eric Clapton	8
3	Garage Inc. (Disc 2)	Metallica	6
4	Acústico	Titãs	5
5	Album Of The Year	Faith No More	5

	Cluster	GenreCount	AvgMinutes	NumberOfCustomers
1	1	8.78	5.66	18
2	2	8.9	11.67	10
3	3	6.23	4.85	31

Hình 8.7. Chọn mô hình album và bấm Load Model (thanh trạng thái hiện Trained/kmeans_album_model.zip): khách 12 thuộc cụm 2, danh sách gợi ý trùng với kết quả ở Hình 7.4.



Hình 8.8. Mô hình ba thuộc tính (K = 4) hiển thị bằng biểu đồ 3D ngay trong cửa sổ.

CHƯƠNG 9. NHẬN XÉT, ĐỀ XUẤT KINH DOANH VÀ HẠN CHẾ

Kết quả gom cụm cho thấy khách Chinook không khác nhau ở lượng mua mà khác nhau ở cách mua. Nhóm lớn nhất tập trung vào ít thể loại, chủ yếu là Rock. Một nhóm nhỏ mua nội dung video và chi tiêu cao nhất. Một nhóm nhỏ khác chủ động tìm album ít người biết. Mỗi nhóm cần một cách tiếp cận riêng.

9.1. Đề xuất theo từng phân khúc

Phân khúc (thực nghiệm, cụm)	**Số khách**	**Đặc điểm dựa trên số liệu**	**Đề xuất**
Khách mua nội dung video (TN1, cụm 2)	10	19,5% lượt mua là video; chi tiêu TB 45,02 USD, cao hơn hai cụm kia 6–7 USD	Bán trọn mùa TV Shows/Drama theo gói, ưu tiên thông báo khi có mùa phim mới; đây là nhóm có giá trị đơn hàng cao nhất
Nghe đa dạng thể loại (TN1, cụm 1)	18	8–10 thể loại; Rock chỉ 31,6%	Danh sách phát pha trộn nhiều thể loại, giới thiệu thể loại khách chưa mua
Tập trung ít thể loại (TN1, cụm 3)	31	4–7 thể loại; Rock 41,7%	Khuyến mãi theo đúng thể loại quen thuộc, tránh gửi nội dung lệch gu
Theo album được nhiều người mua (TN2, cụm 2)	25	Ít album nhất (18–22), độ phổ biến cao nhất (5,56)	Gợi ý các album "bán chạy trong nhóm" như ở Chương 7; nhóm này dễ phản hồi với bảng xếp hạng
Săn album ít người biết (TN2, cụm 3)	7	23–27 album, độ phổ biến thấp nhất (4,19); Classical 9,4%	Giới thiệu album mới phát hành, album ít người mua, dòng nhạc Classical; không dùng gợi ý theo mức phổ biến
Ít thể loại nhưng nhiều album (TN3, cụm 4)	16	6,6 thể loại, 24,4 album; Rock 42,3%, Metal 15,3%	Gợi ý album khác cùng thể loại Rock/Metal, ví dụ các album còn lại của nghệ sĩ đã mua

Về vận hành, mô hình đã lưu cho phép xếp một khách mới vào phân khúc ngay khi có lịch sử mua, chỉ cần hai con số GenreCount và AvgMinutes (Chương 7). Theo bài giảng

08, khi khách tiếp tục mua, dữ liệu mới được ghi vào cơ sở dữ liệu và mô hình cần được huấn luyện lại định kỳ, vì sở thích của khách thay đổi theo thời gian.

Người dùng kết quả không cần chạy code. Ba file Excel trong results/excel/ cho sẵn danh sách khách của từng phân khúc, có email và quốc gia, để lọc và gửi chương trình khuyến mãi theo nhóm. Trang results/html/index.html cho biết mỗi nhóm mua thể loại và album nào, và tab tra cứu cho ngay 5 album nên giới thiệu cho từng khách.

9.2. Hạn chế

1. **Dữ liệu mẫu và nhỏ.** Chỉ có 59 khách, hành vi mua được sinh khá đều. Các cụm có từ 7 đến 31 khách, nên chỉ cần vài khách thay đổi hành vi là tâm cụm đã dịch chuyển đáng kể.
2. **Mức tách cụm trung bình.** Silhouette 0,31–0,43 cho thấy ranh giới giữa các cụm chưa sắc nét. Khách nằm gần biên, ví dụ khách có GenreCount = 7 và AvgMinutes ≈ 8 phút, có thể chuyển cụm khi dữ liệu thay đổi nhỏ.
3. **Thuộc tính rời rạc.** GenreCount và AlbumCount chỉ nhận số nguyên trong khoảng hẹp. Nhiều khách trùng tọa độ, các cụm nằm sát nhau theo từng vạch giá trị.
4. **AlbumPopularity có tính cả chính khách đang xét.** Độ phổ biến của mỗi album mà khách đã mua luôn ít nhất là 1. Ảnh hưởng nhỏ và như nhau với mọi khách, nhưng cách tính chặt chẽ hơn là trừ khách đó ra.
5. **Gợi ý album chưa được đo độ chính xác.** Phương pháp gợi ý dựa trên mức phổ biến trong cụm, và bài làm chưa kiểm tra xem khách có thật sự mua các album được gợi ý hay không.
6. **K-Means giả định cụm có dạng cầu, kích thước tương đương.** Cụm 2 của thực nghiệm 1 trải dài theo trục thời lượng (9–16,55 phút), không có dạng cầu.

9.3. Hướng phát triển

- So sánh K-Means với Agglomerative Clustering và DBSCAN, hai giải thuật được nêu trong phần thảo luận của bài giảng 07. DBSCAN không cần chọn trước K và xử lý được cụm không có dạng cầu như cụm video.
- Đánh giá gợi ý album: ẩn hóa đơn cuối cùng của mỗi khách, huấn luyện trên phần còn lại, rồi đo tỷ lệ album gợi ý xuất hiện trong hóa đơn bị ẩn.
- Bổ sung thuộc tính theo thời gian, như số ngày từ lần mua gần nhất hay tần suất mua theo quý (hướng RFM), khi có dữ liệu thực tế đa dạng hơn.
- Đưa thuật toán Kneedle vào nút Elbow của giao diện. Hiện nút này chỉ vẽ đường WCSS và người dùng tự chọn K bằng spinBoxK; có thể gọi findElbow để tô khoảng elbow và điền sẵn K đề xuất.
- Tính tay một vòng lặp K-Means trên Excel cho 4–5 khách, theo cách làm của KMeans_Bai1.xlsx, để đối chiếu với kết quả của scikit-learn.

9.4. Kết luận

Bài làm xây dựng đầy đủ một hệ thống gom cụm khách hàng trên dữ liệu Chinook, từ khám phá dữ liệu đến giao diện sử dụng mô hình. Đóng góp chính nằm ở bốn thuộc tính tự đề xuất. Bốn thuộc tính này biến dữ liệu mua hàng rất thưa (1,9% ô khác 0) thành biểu diễn gom cụm được, nâng Silhouette từ khoảng 0,03 (ma trận khách × album) lên 0,43. Số cụm của cả ba thực nghiệm được chọn tự động bằng thuật toán Kneedle và trùng với kết quả đọc đồ thị Elbow kết hợp Silhouette. Mô hình track với K = 3 cho kết quả tốt nhất về độ tách cụm. Mô hình album phục vụ trực tiếp việc gợi ý. Mô hình ba thuộc tính cho phân khúc chi tiết nhất. Kết quả được đưa ra ba dạng dùng được ngay: file Excel theo từng cụm, trang HTML tổng hợp và giao diện PyQt6.